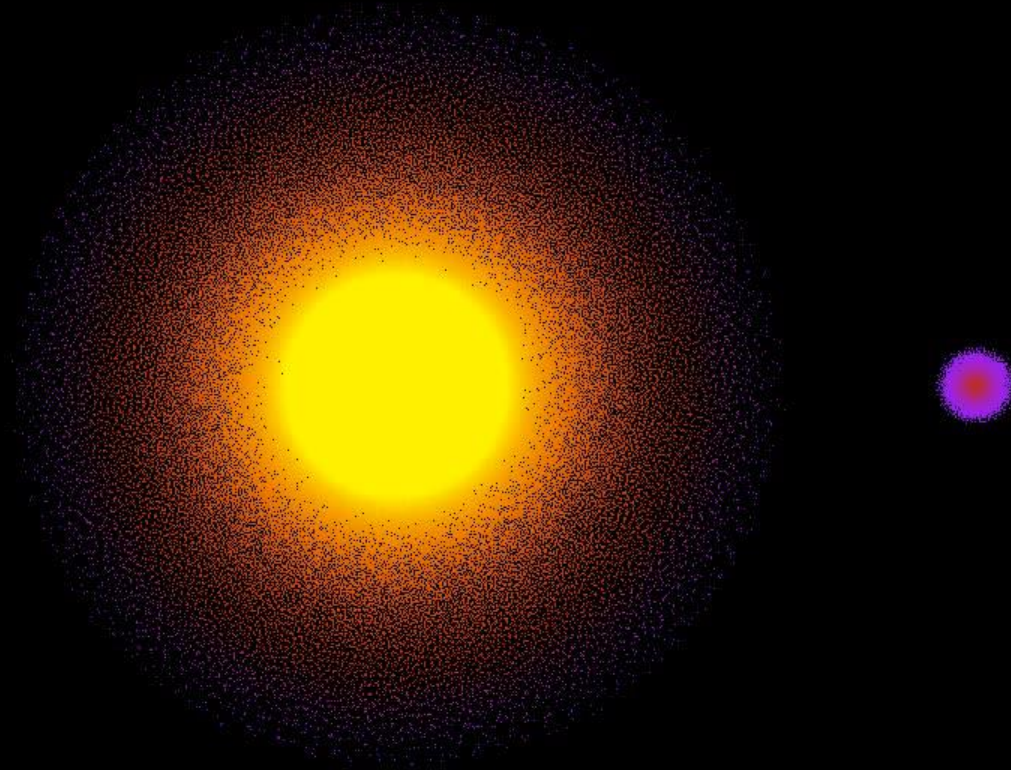


Introduction to Parallel Computing



Dr. Rubén M. Cabezón
Dr. Aurélien Cavelan

sciCORE
Center for scientific computing



University
of Basel

OpenMP

Outline

Introduction

OpenMP

- ☐ Scheduling
- ☐ Variable scope
- ☐ Reduction
- ☐ Atomic
- ☐ Collapse
- ☐ Orphaned directives

Python

- ☐ Multiprocessing
- ☐ Numba

OpenMP and GPU offloading

MPI (just a smidge)

Launching in clusters (SLURM)



09:00 – 10:30: Lecture

10:30 – 10:45: Coffee break

10:45 – 12:00: Lecture

12:00 – 13:30: Lunch

13:30 – 15:00: Lecture

15:00 – 15:15: Coffee break

15:15 – 17:00: Lecture



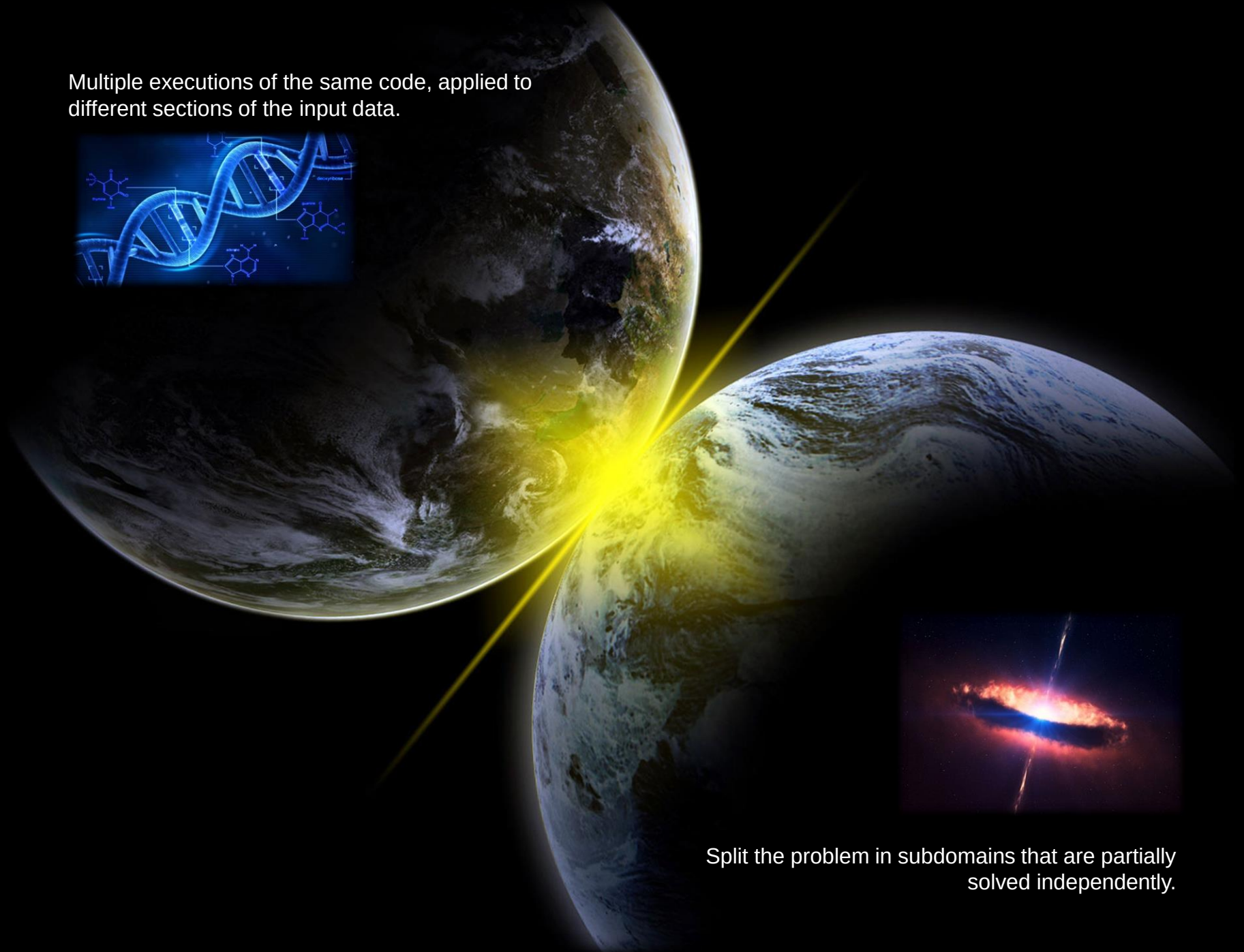
First thing to do:

Download the slides of the course:

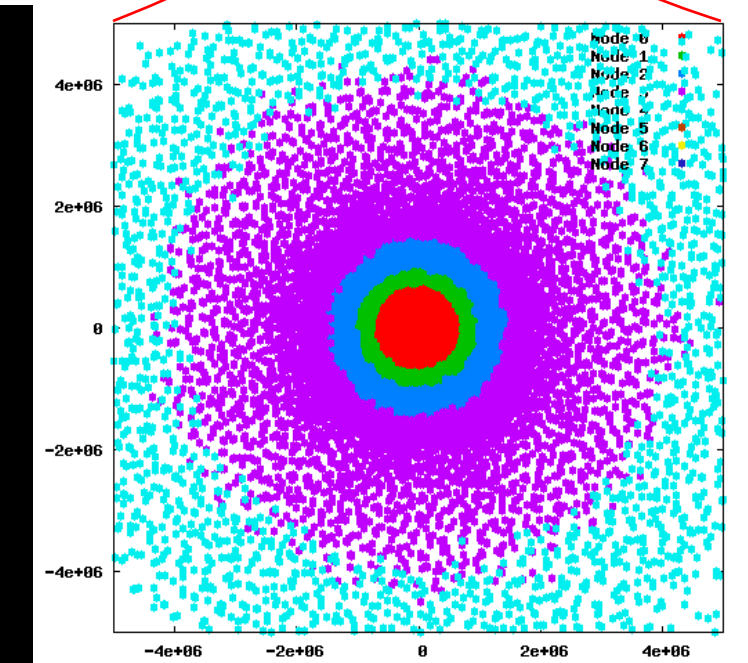
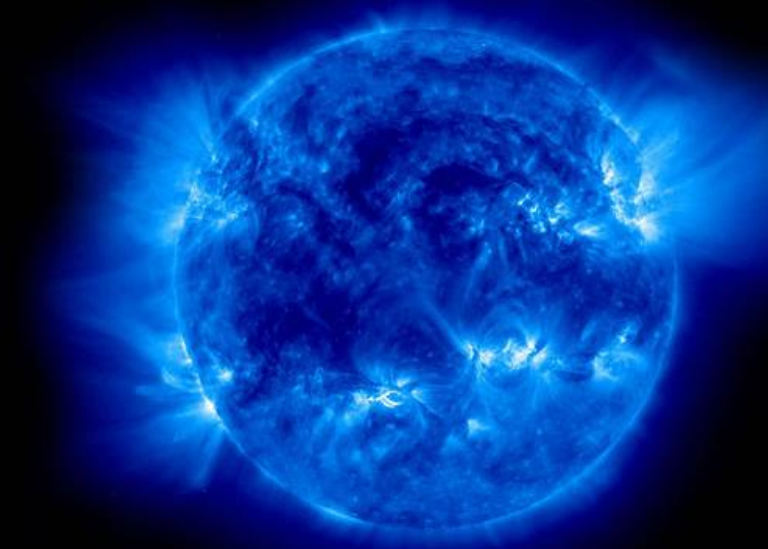
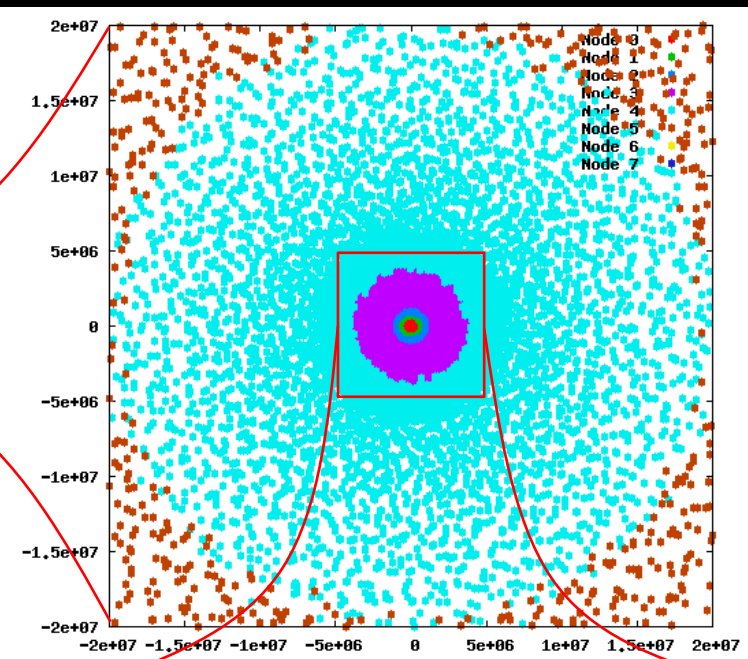
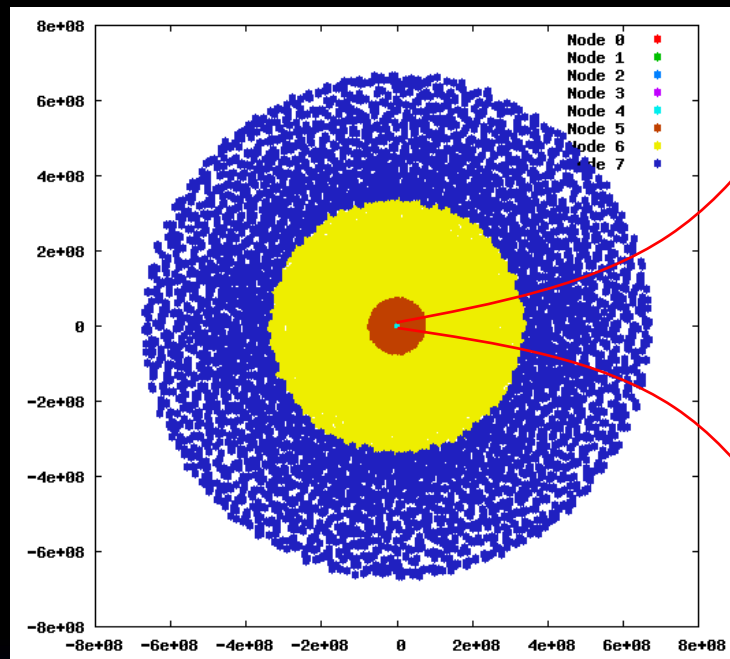
From the browser: https://bit.ly/scicore_openmp

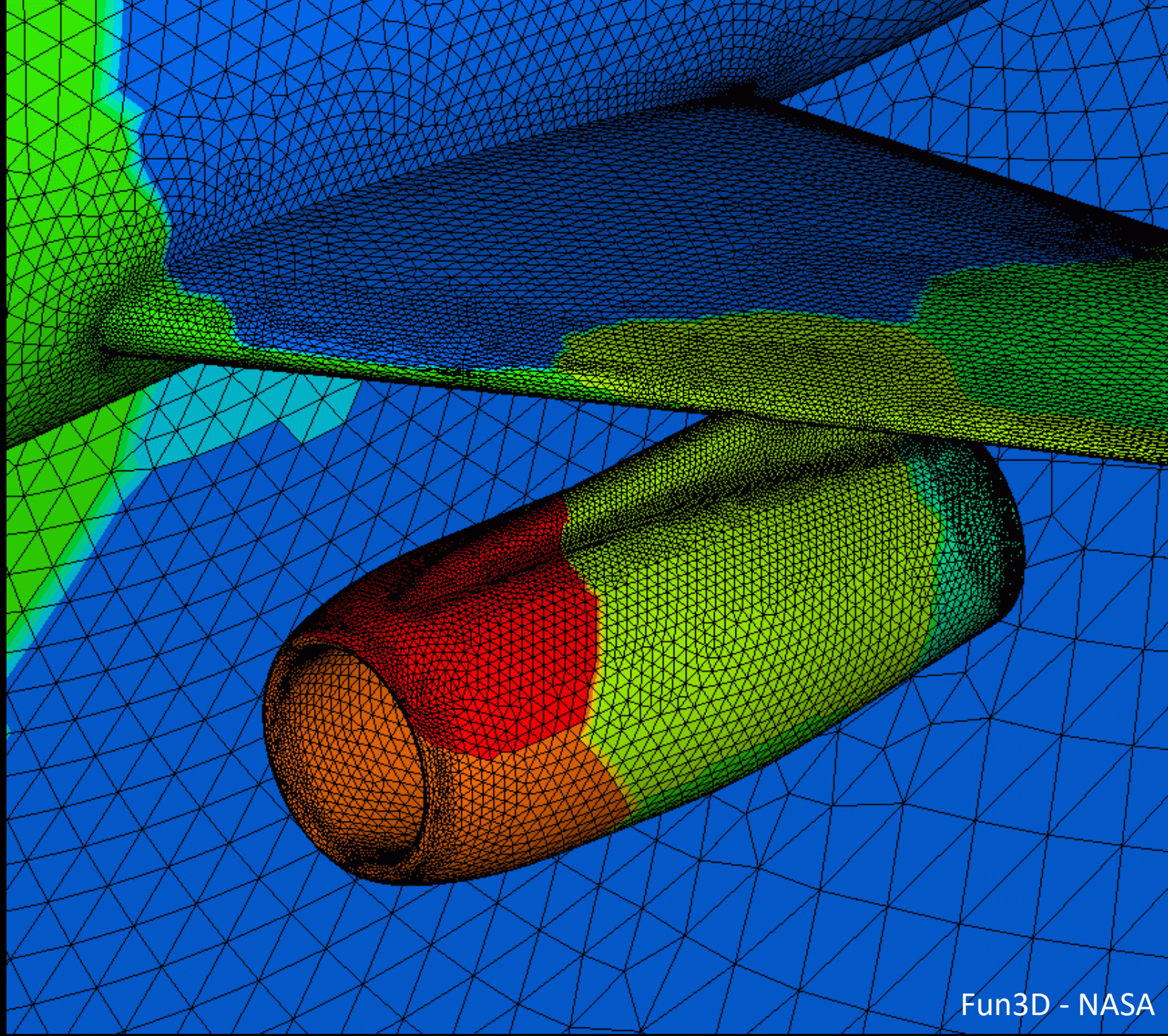
You can find there the slides of the course (exercises are redacted)

Multiple executions of the same code, applied to different sections of the input data.



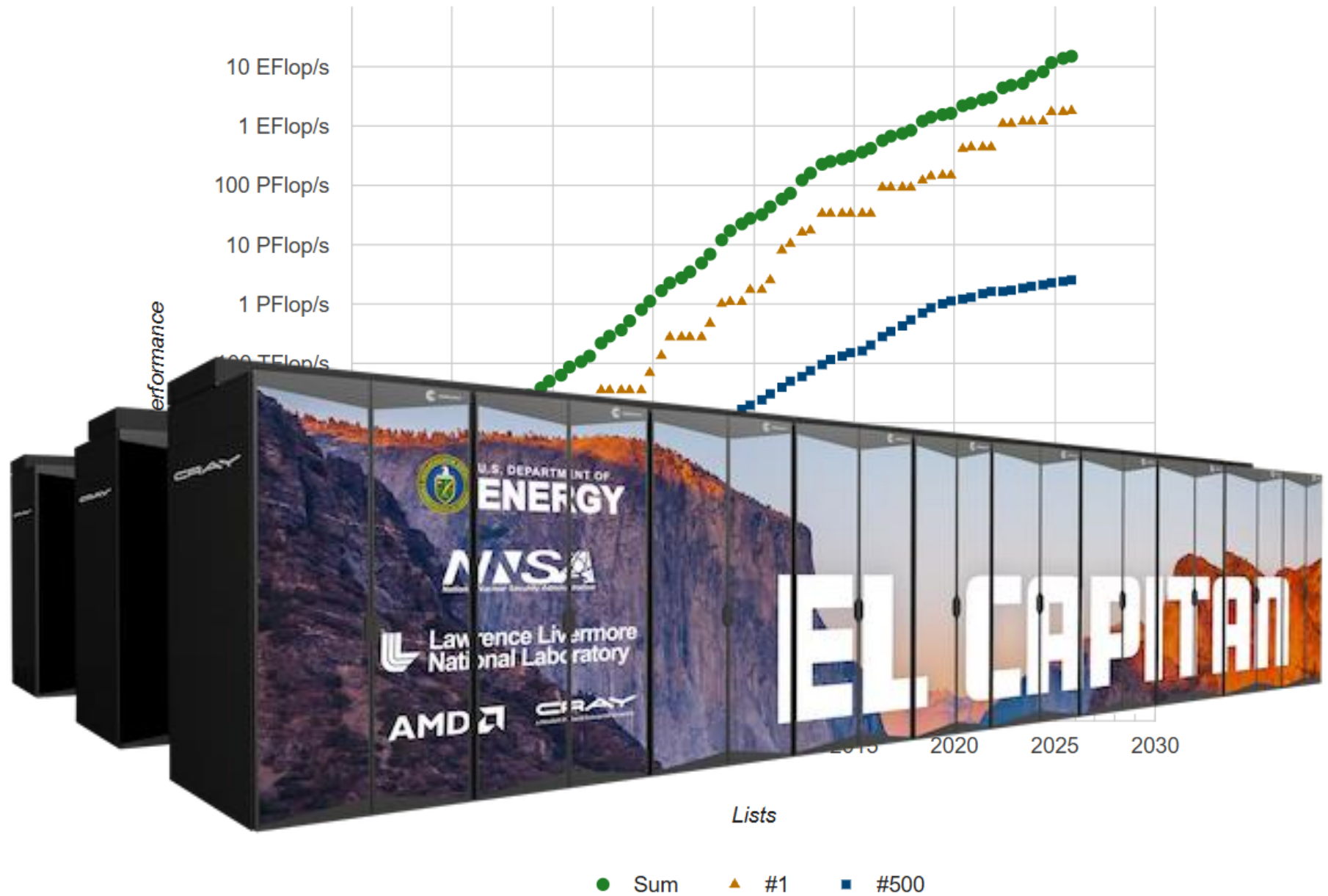
Split the problem in subdomains that are partially solved independently.

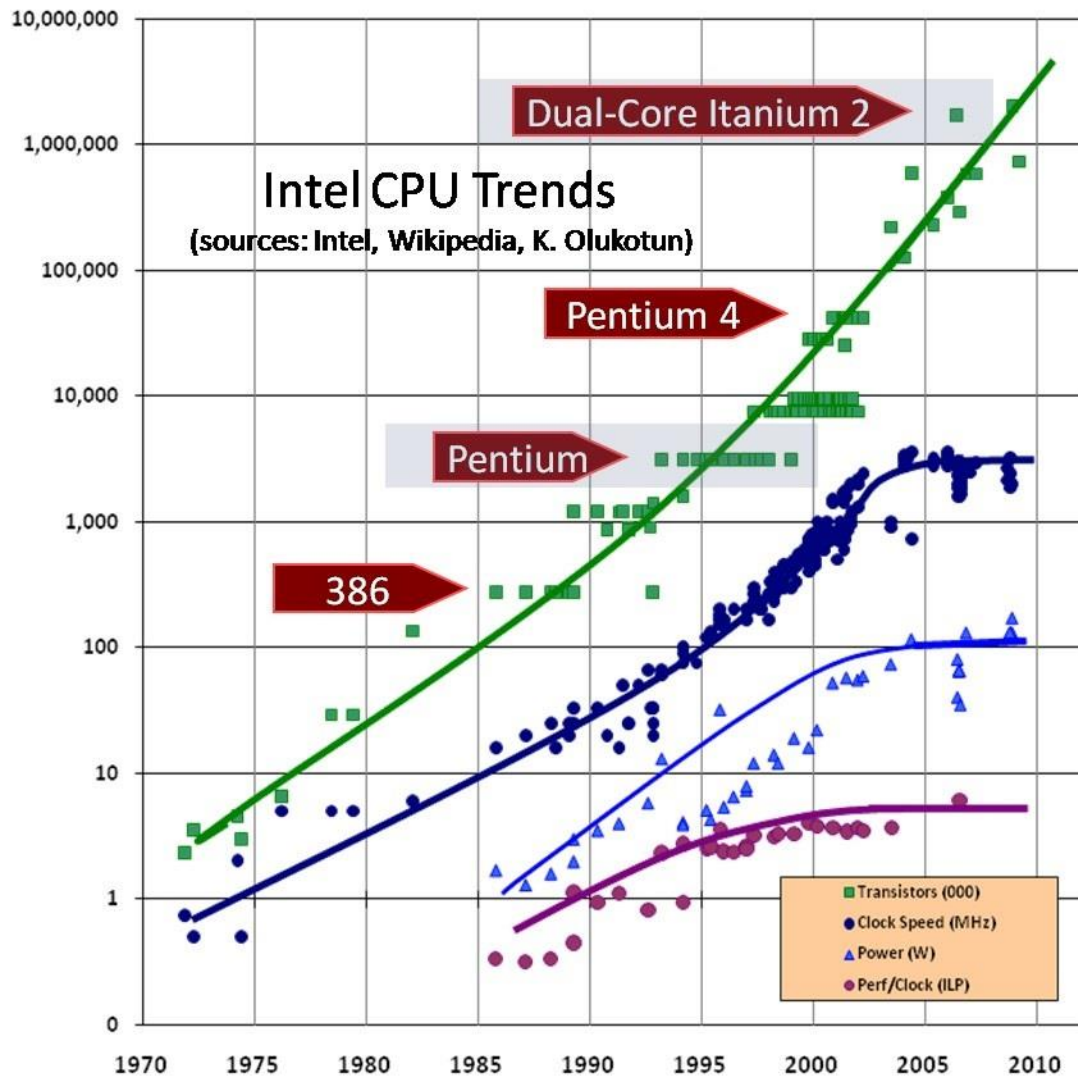




Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,340,000	1,809.00	2,821.10	29,685
2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	1,000.00	1,226.28	15,794
5	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure United States	2,073,600	561.20	846.84	
6	HPC6 - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, RHEL 8.9, HPE Eni S.p.A. Italy	3,143,520	477.90	606.97	8,461
7	Supercomputer Fugaku - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu RIKEN Center for Computational Science Japan	7,630,848	442.01	537.21	29,899
8	Alps - HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cray OS, HPE Swiss National Supercomputing Centre (CSCS) Switzerland	2,121,600	434.90	574.84	7,124

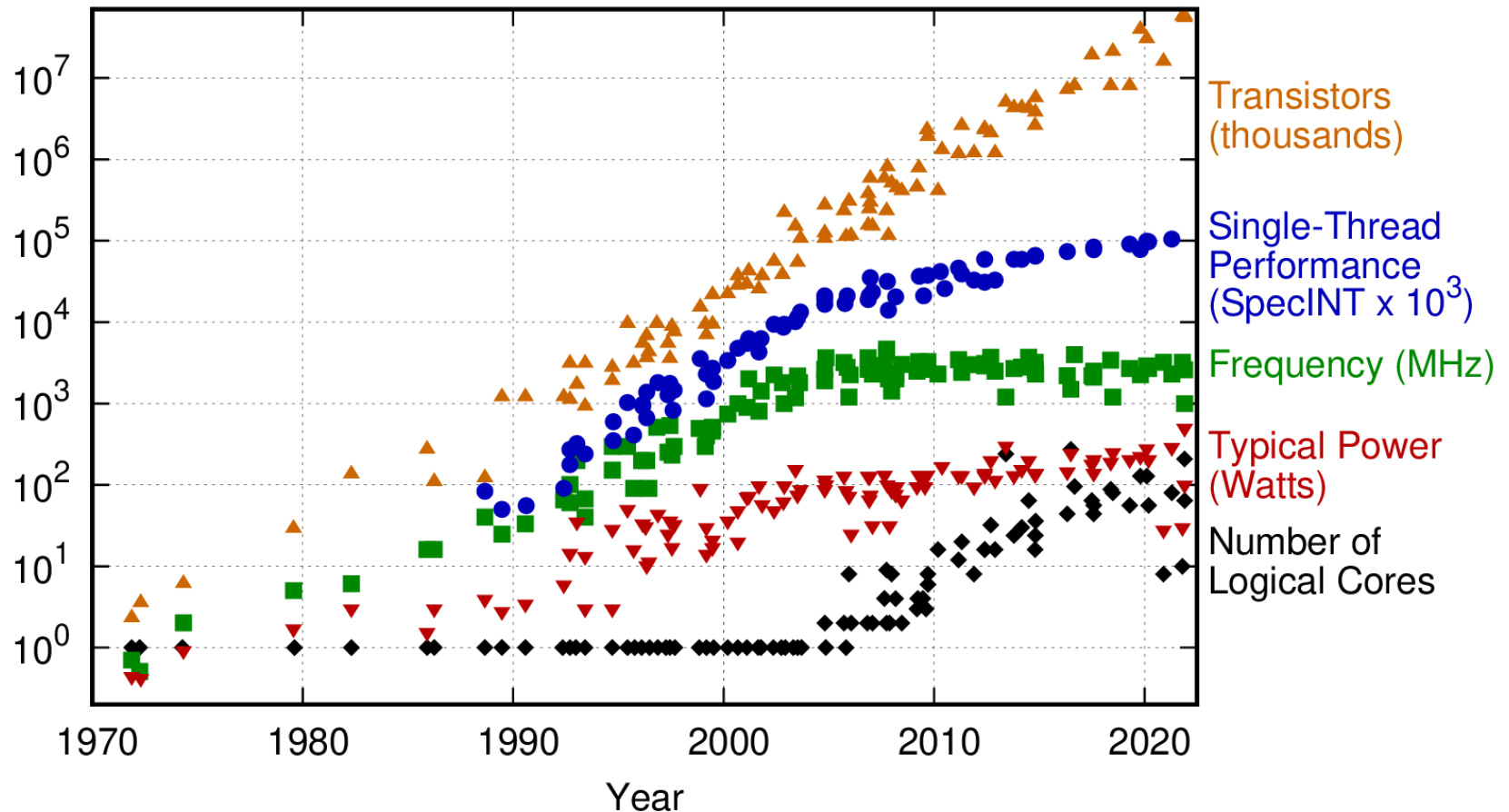
Performance Development





More transistors do not mean faster anymore!

50 Years of Microprocessor Trend Data



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

In general, the problem can be reduced to a logical repetitive structure:

```
do i = 1, n  
<calculations>  
end do
```

Are these <calculations> completely **independent** from each other?



Yes

No

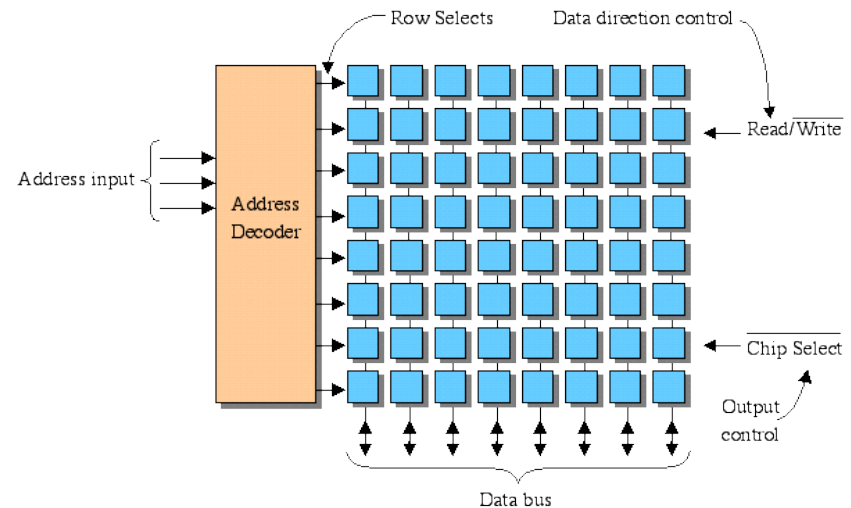
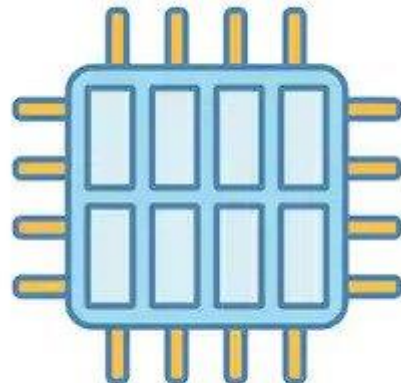
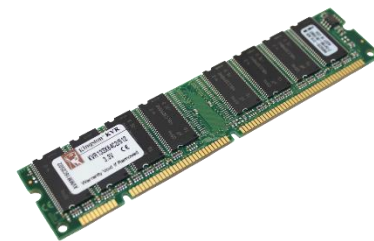
- What's the degree of dependency?
- Can I rewrite/factorize to eliminate the dependency?
- Do variables share contents? Is it necessary?





Open Multi-Processing

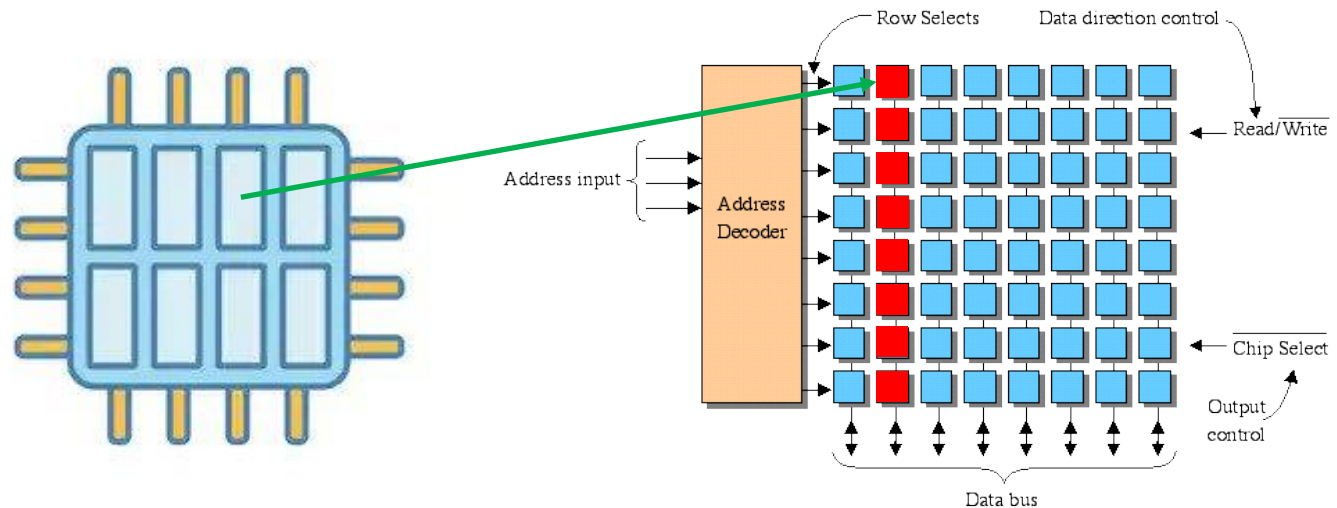
Works in **shared** memory systems
Can use as many cores as there are in the **compute node**
Uses **threads** to split the work
Threads have both **private** and **shared** variables
Uses **preprocessor directives** in commented lines





Open Multi-Processing

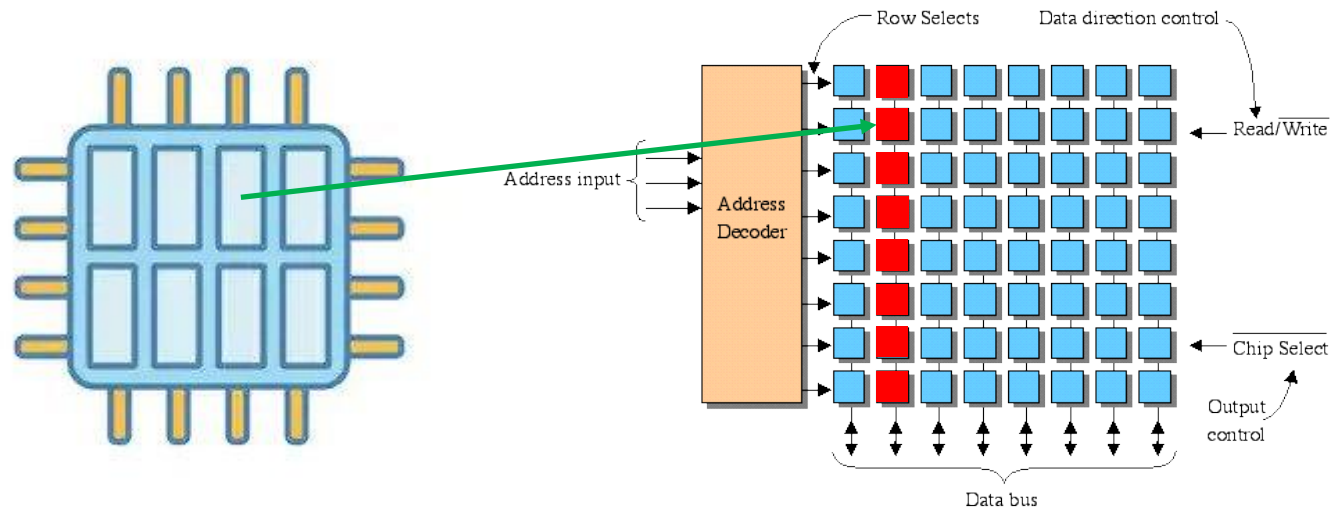
Works in **shared** memory systems
Can use as many cores as there are in the **compute node**
Uses **threads** to split the work
Threads have both **private** and **shared** variables
Uses **preprocessor directives** in commented lines





Open Multi-Processing

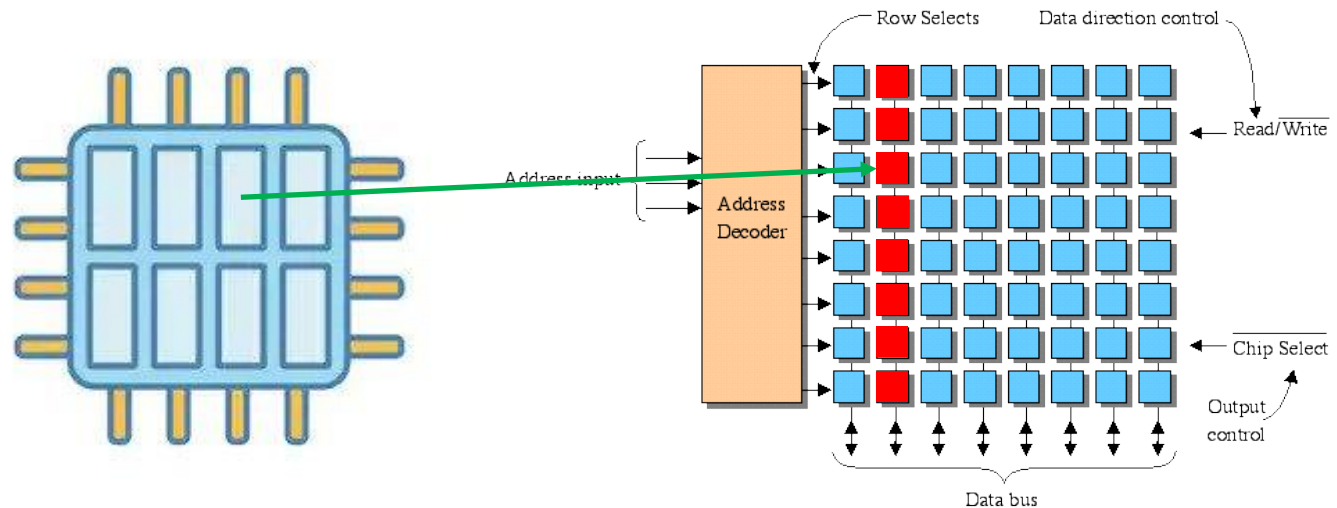
Works in **shared** memory systems
Can use as many cores as there are in the **compute node**
Uses **threads** to split the work
Threads have both **private** and **shared** variables
Uses **preprocessor directives** in commented lines





Open Multi-Processing

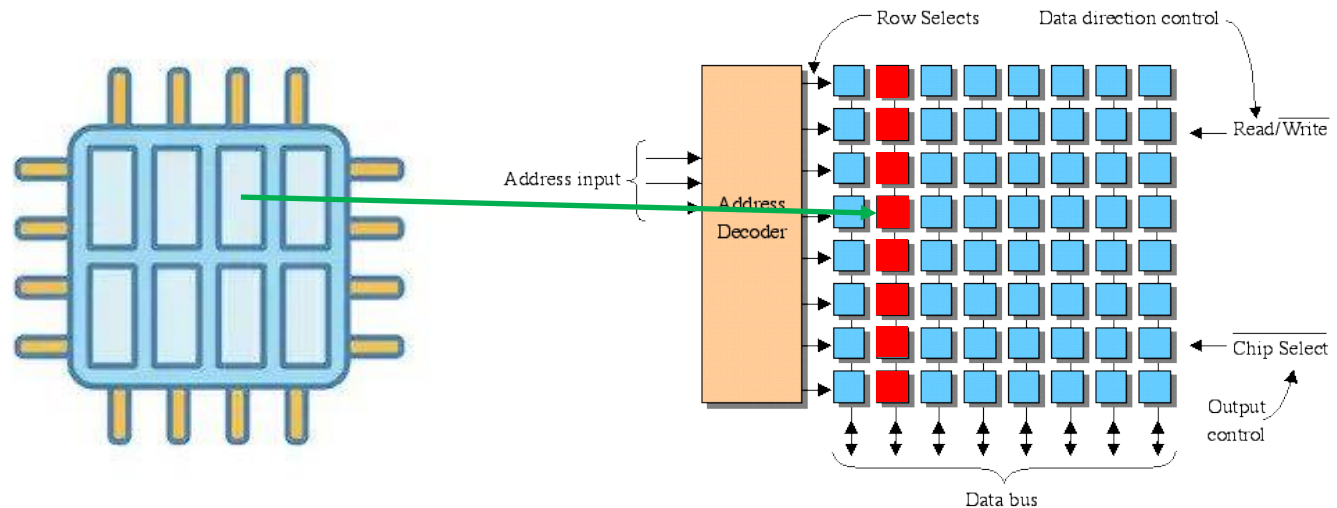
Works in **shared** memory systems
Can use as many cores as there are in the **compute node**
Uses **threads** to split the work
Threads have both **private** and **shared** variables
Uses **preprocessor directives** in commented lines





Open Multi-Processing

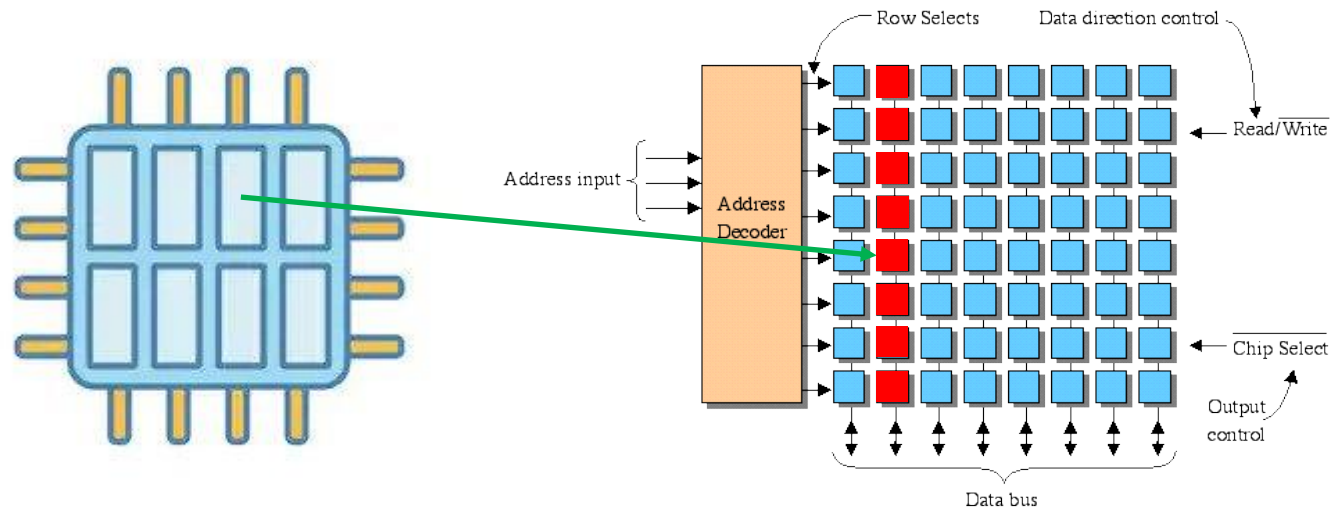
Works in **shared** memory systems
Can use as many cores as there are in the **compute node**
Uses **threads** to split the work
Threads have both **private** and **shared** variables
Uses **preprocessor directives** in commented lines





Open Multi-Processing

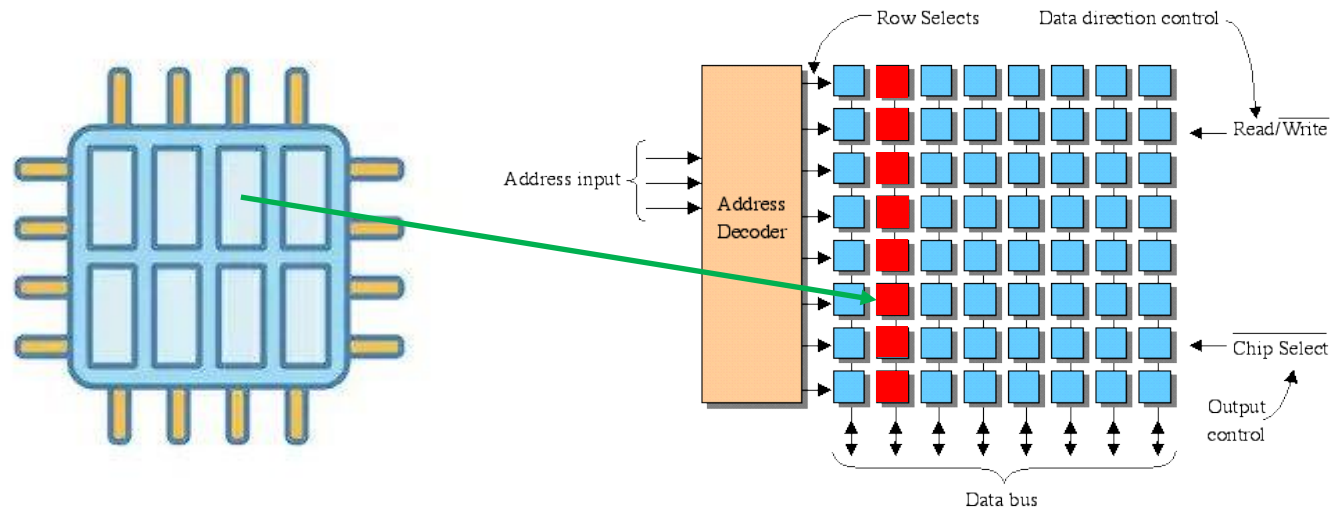
Works in **shared** memory systems
Can use as many cores as there are in the **compute node**
Uses **threads** to split the work
Threads have both **private** and **shared** variables
Uses **preprocessor directives** in commented lines





Open Multi-Processing

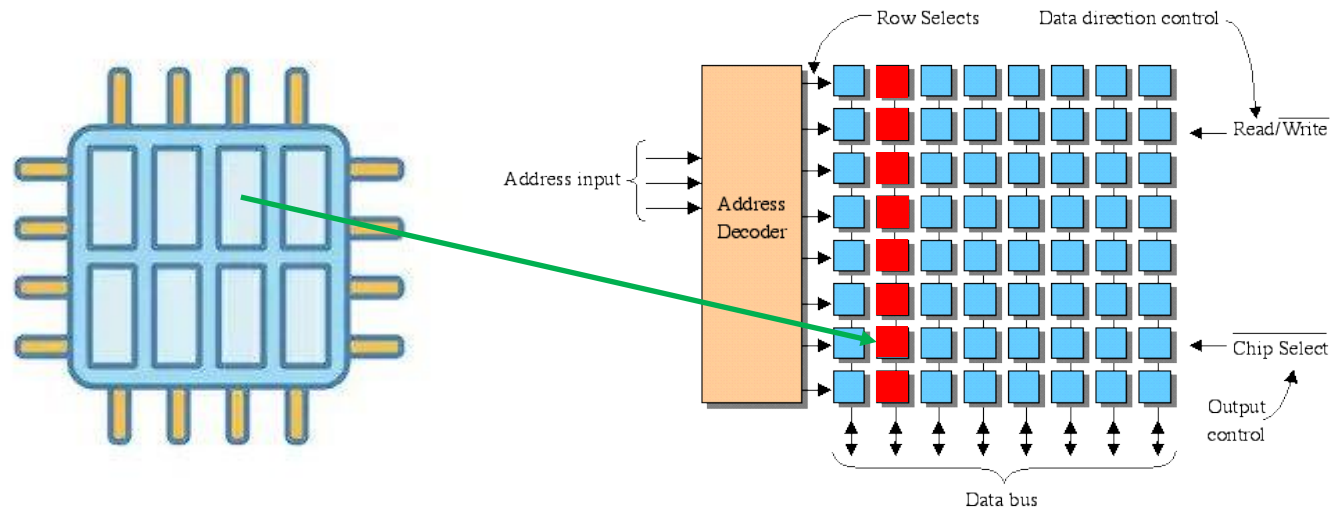
Works in **shared** memory systems
Can use as many cores as there are in the **compute node**
Uses **threads** to split the work
Threads have both **private** and **shared** variables
Uses **preprocessor directives** in commented lines





Open Multi-Processing

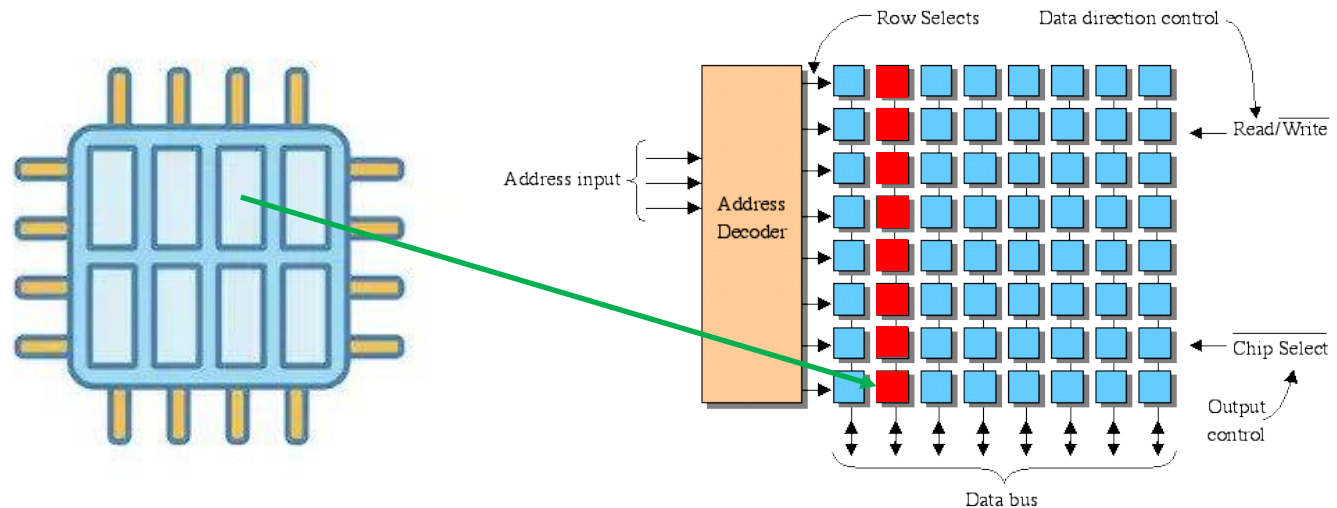
Works in **shared** memory systems
Can use as many cores as there are in the **compute node**
Uses **threads** to split the work
Threads have both **private** and **shared** variables
Uses **preprocessor directives** in commented lines





Open Multi-Processing

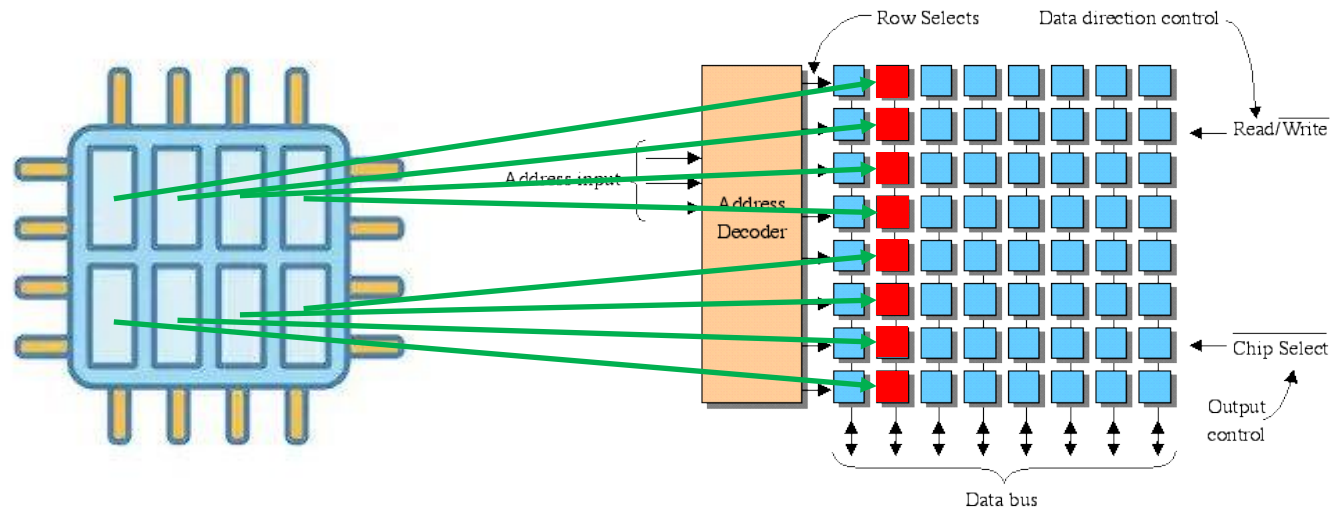
Works in **shared** memory systems
Can use as many cores as there are in the **compute node**
Uses **threads** to split the work
Threads have both **private** and **shared** variables
Uses **preprocessor directives** in commented lines





Open Multi-Processing

Works in **shared** memory systems
Can use as many cores as there are in the **compute node**
Uses **threads** to split the work
Threads have both **private** and **shared** variables
Uses **preprocessor directives** in commented lines



Some resources

- Main references:

<https://www.openmp.org/wp-content/uploads/OpenMP-RefGuide-6.0-OMP60SC24-web.pdf>

- Examples & tutorials:

<https://www.openmp.org/wp-content/uploads/openmp-examples-6.0.pdf>

<https://hpc-tutorials.llnl.gov/openmp/>

<https://www.youtube.com/playlist?list=PLLX-Q6B8xqZ8n8bwjGdzBJ25X2utwnoEG>

<https://numba.pydata.org/numba-doc/dev/user/5minguide.html>

<https://mpi4py.readthedocs.io/en/stable/tutorial.html>

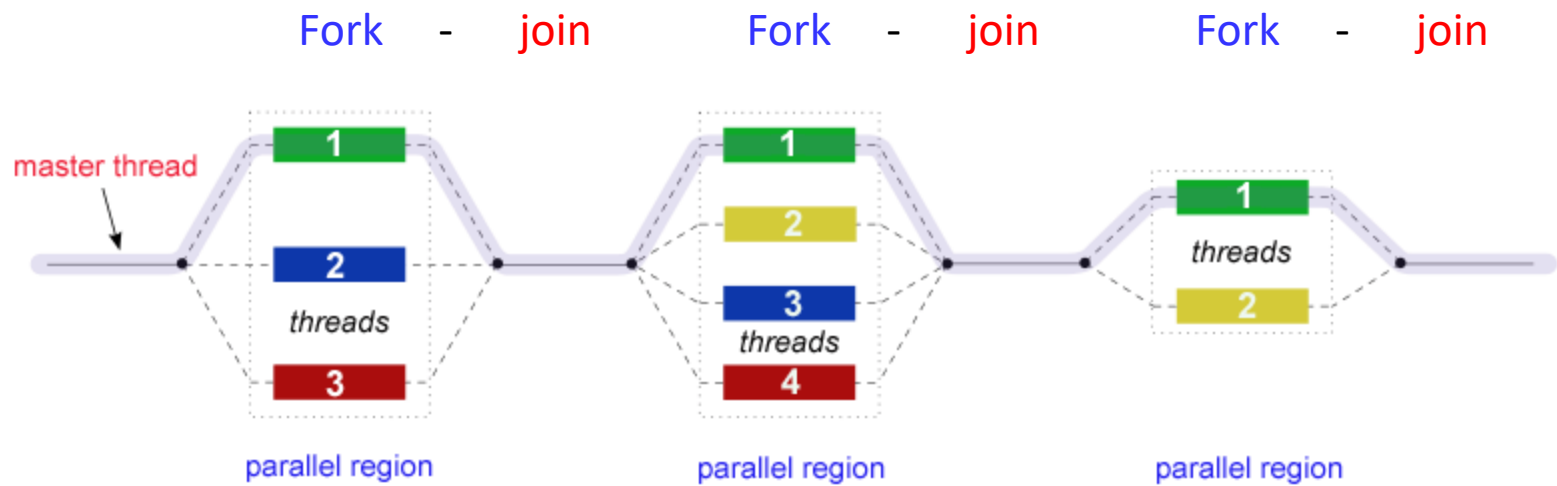
- Books:

The OpenMP Common Core: Making OpenMP Simple Again (Mattson, et al. 2019)

- In-person Courses:

Unibas Master and PhD course, <https://hpc.dmi.unibas.ch/HPC/Teaching.html>

In particular: Foundations of Distributed Systems (45402-01/HS): F. Ciorba, H. Schuldt, C. Tschundin
High-Performance Computing (17164-01/FS): F. Ciorba



FORTRAN

```
use omp_lib

!$omp parallel
...
!$omp do
do ...
enddo
!$omp end do
...
!$omp end parallel
```

C/C++

```
#include <omp.h>

#pragma omp parallel
{
    ...
    #pragma omp for
    for(...){
        ...
    }
}
```

- Serial and parallel program share the same source code.
- Serial compiler simply overlooks parallel directives (they are comments).
- Modifying parallel directives cannot break the serial code.
- Modifying the serial code outside of parallel regions cannot break the parallelization.
- Easy to maintain.
- Compact code.

```
!$omp parallel private(id)
id=omp_get_thread_num()
print *, 'I am thread: ', id
!$omp end parallel
```

Runtime library routine.

- omp_get_num_threads()
- omp_set_num_threads()
- omp_get_wtime()
- ...

```
I am thread 0
I am thread 1
I am thread 2
I am thread 3
```

```
I am thread 0
I am thread 3
I am thread 2
I am thread 1
```

```
I am thread 2
I am thread 1
I am thread 0
I am thread 3
```

```
I am thread 0
I am thread 1
I am thread 2
I am thread 3
```

We cannot ensure that the threads will execute and finish in order!

```
do i = 1, n  
    f(i) = sin(dble(i))  
end do
```

f(i) =

sin(1)
sin(2)
sin(3)
sin(4)
sin(5)
sin(6)
sin(7)
sin(8)

```
!$omp parallel  
!$omp do  
do i = 1, n  
    f(i) = sin(dble(i))  
end do  
!$omp end do  
!$omp end parallel
```

f(i) =

sin(1)
sin(2)
sin(3)
sin(4)
sin(5)
sin(6)
sin(7)
sin(8)

```
do i = 1, n  
    f(i) = sin(dble(i))  
end do
```

f(i) =

sin(1)
sin(2)
sin(3)
sin(4)
sin(5)
sin(6)
sin(7)
sin(8)

Check env. variable, f.e.:
\$OMP_NUM_THREADS=4

```
!$omp parallel  
!$omp do  
do i = 1, n  
    f(i) = sin(dble(i))  
end do  
!$omp end do  
!$omp end parallel
```

f(i) =

sin(1)
sin(2)
sin(3)
sin(4)
sin(5)
sin(6)
sin(7)
sin(8)

```
do i = 1, n  
    f(i) = sin(dble(i))  
end do
```

```
!$omp parallel  
!$omp do  
do i = 1, n  
    f(i) = sin(dble(i))  
end do  
!$omp end do  
!$omp end parallel
```

f(i) =

sin(1)
sin(2)
sin(3)
sin(4)
sin(5)
sin(6)
sin(7)
sin(8)

Check env. variable, f.e.:
\$OMP_NUM_THREADS=4

f(i) =

sin(1)
sin(2)
sin(3)
sin(4)
sin(5)
sin(6)
sin(7)
sin(8)

Creates 4 threads and distributes the work evenly

```
do i = 1, n  
    f(i) = sin(dble(i))  
end do
```

```
!$omp parallel  
!$omp do  
do i = 1, n  
    f(i) = sin(dble(i))  
end do  
!$omp end do  
!$omp end parallel
```

f(i) =

sin(1)
sin(2)
sin(3)
sin(4)
sin(5)
sin(6)
sin(7)
sin(8)

Check env. variable, f.e.:
\$OMP_NUM_THREADS=4

f(i) =

sin(1)	← thread 0
sin(2)	← thread 0
sin(3)	← thread 1
sin(4)	← thread 1
sin(5)	← thread 2
sin(6)	← thread 2
sin(7)	← thread 3
sin(8)	← thread 3

**x4
Speed-up!**

Creates 4 threads and distributes the work evenly

```

!$omp parallel
!$omp do
do i = 1, n

    f(i) = sin(dble(i))

end do
!$omp end do
!$omp end parallel

```

Establish parallel section

Parallel loop evenly divided among the threads

The number of threads is given by the environment variable: `$OMP_NUM_THREADS`
 If not defined, OpenMP assumes `OMP_NUM_THREADS = # cores in the computer`.
 Can be changed with: `export OMP_NUM_THREADS = <integer>`
 (consider adding it to `.bashrc`)

These are comments!

They are understood by the compiler only when compiled with `-openmp` option (or similar). Otherwise they are ignored.

An OpenMP calculation will appear as using more than 100% of CPU

cabezon@kb-kbts01-pdl15:~

File Edit View Search Terminal Help

top - 13:02:21 up 19 days, 21:27, 9 users, load average: 0.58, 0.20, 0.13
 Tasks: 383 total, 2 running, 381 sleeping, 0 stopped, 0 zombie
 Cpu(s): 99.5%us, 0.5%sy, 0.0%ni, 0.0%id, 0.0%wa, 0.0%hi, 0.0%si, 0.0%st
 Mem: 7965056k total, 7768652k used, 196404k free, 14600k buffers
 Swap: 2064380k total, 1725328k used, 339052k free, 813432k cached

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
23339	cabezon	20	0	89876	904	696	R	769.6	0.0	0:11.19	integral.out
31848	cabezon	20	0	1135m	151m	11m	S	17.8	2.0	318:52.56	chrome
21008	cabezon	20	0	3985m	2.1g	2.1g	S	5.9	27.7	10:53.79	VirtualBox
19624	cabezon	20	0	956m	118m	8828	S	2.0	1.5	241:53.73	chrome
23309	cabezon	20	0	30816	1876	1276	R	2.0	0.0	0:00.32	top
1	root	20	0	36924	588	332	S	0.0	0.0	0:01.17	init
2	root	20	0	0	0	0	S	0.0	0.0	0:00.07	kthreadd
3	root	RT	0	0	0	0	S	0.0	0.0	0:01.64	migration/0
4	root	20	0	0	0	0	S	0.0	0.0	0:05.95	ksoftirqd/0
5	root	RT	0	0	0	0	S	0.0	0.0	0:00.00	stopper/0
6	root	RT	0	0	0	0	S	0.0	0.0	0:01.97	watchdog/0
7	root	RT	0	0	0	0	S	0.0	0.0	0:00.97	migration/1
8	root	RT	0	0	0	0	S	0.0	0.0	0:00.00	stopper/1
9	root	20	0	0	0	0	S	0.0	0.0	0:04.68	ksoftirqd/1
10	root	RT	0	0	0	0	S	0.0	0.0	0:01.78	watchdog/1
11	root	RT	0	0	0	0	S	0.0	0.0	0:00.62	migration/2
12	root	RT	0	0	0	0	S	0.0	0.0	0:00.00	stopper/2

```

!$omp parallel
!$omp do schedule(static)
do i = 1, n

    f(i) = sin(dble(i))

end do
!$omp end do
!$omp end parallel

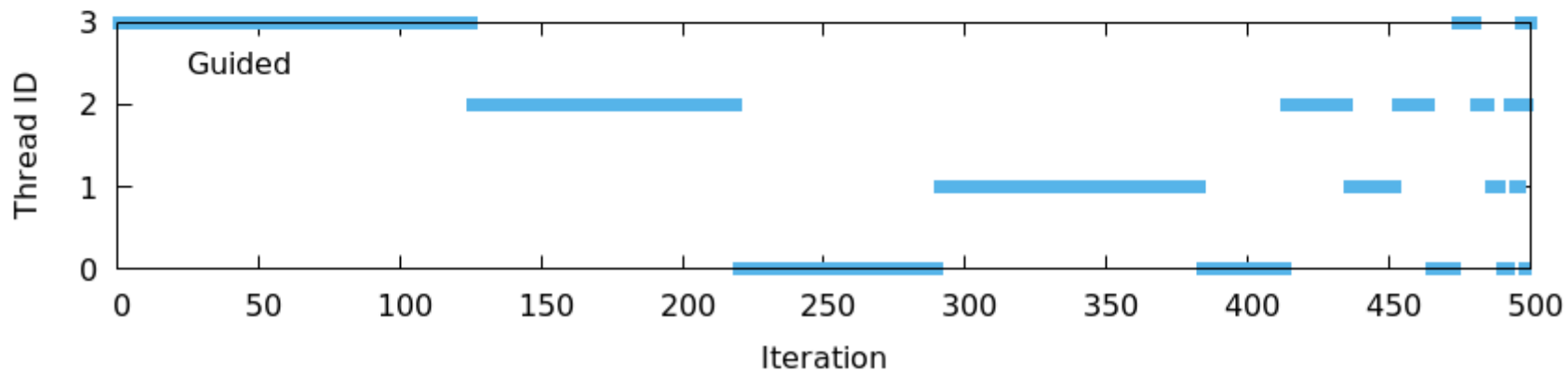
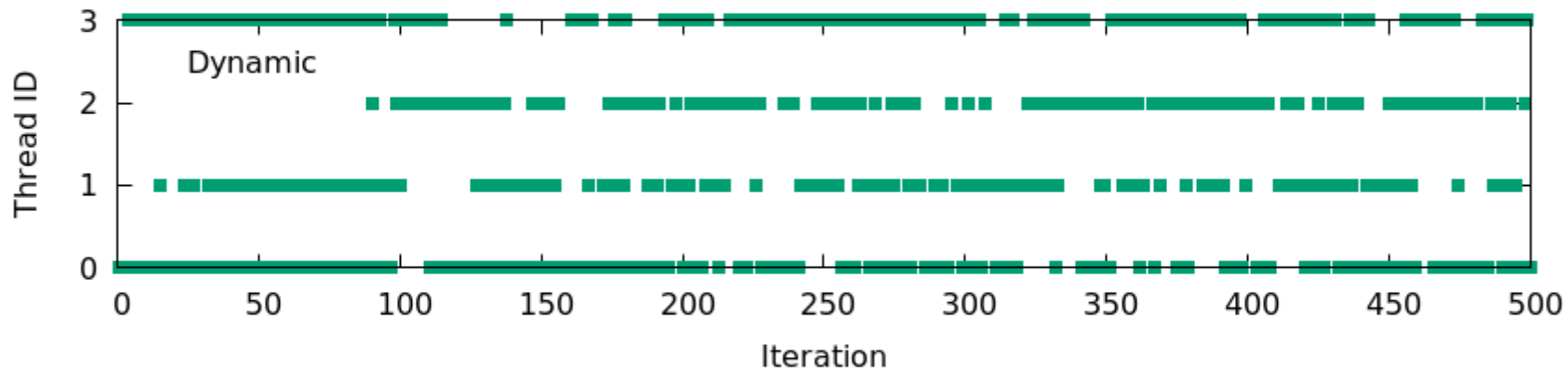
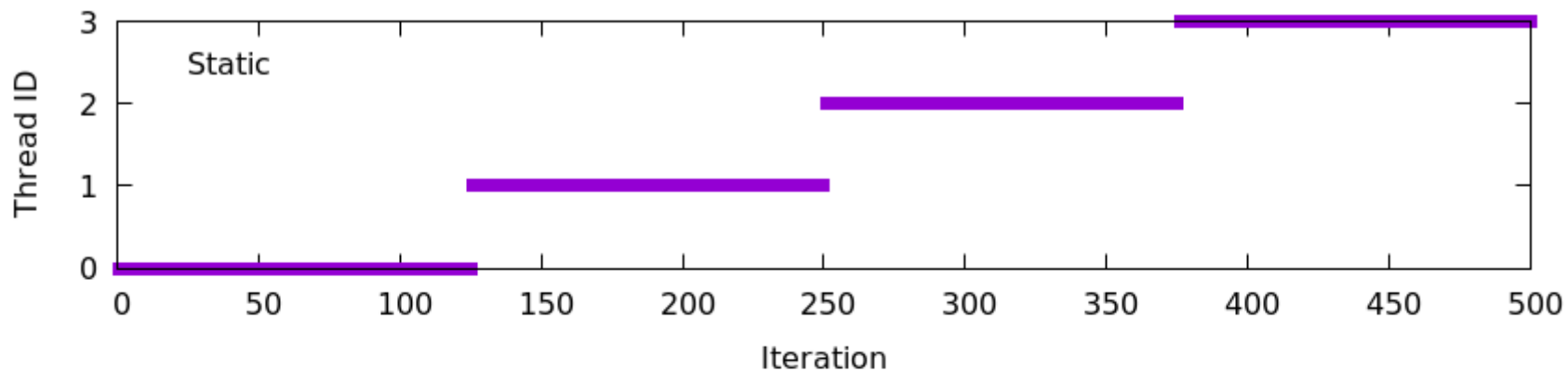
```

Parallel loop **evenly** divided among the threads

$$\text{chunk} = \frac{\text{loop count}}{\text{\# threads}} \quad \text{static schedule}$$

Other options are available: `!$omp do schedule(<kind> [,chunk_size])`

Kind	Description
static	Divides the loop in equal-sized chunks (or as equal as possible).
dynamic	When a thread finishes, it retrieves the next chunk from the internal queue. Be aware of the extra overhead! (default chunk size = 1).
guided	Similar to dynamic, but starts off large and decreases to better handle imbalance. (default chunk size = same as static).
auto	Decision regarding scheduling is delegated to the compiler.
runtime	Uses OMP_SCHEDULE env. Variable to select the scheduling type.



```
factorial(1) = 1
```

```
do i = 2, 6
```

```
    factorial(i) = i * factorial(i-1)
```

```
end do
```

factorial(i) =

1

2

6

24

120

720

5040

```
factorial(1) = 1
```

```
!$omp parallel  
!$omp do schedule(static)  
do i = 2, 6
```

```
    factorial(i) = i * factorial(i-1)
```

```
end do  
!$omp end do  
!$omp end parallel
```

There is something wrong here!

We cannot ensure this!

factorial(i) =

1
2
6
24
120
720
5040

factorial(i) =

1	
2	← thread 0
6	← thread 0
24	← thread 1
120	← thread 1
720	← thread 2
5040	← thread 2

```
factorial(1) = 1
```

```
!$omp parallel  
!$omp do schedule(static)  
do i = 2, 6
```

```
    factorial(i) = i * factorial(i-1)
```

```
end do  
!$omp end do  
!$omp end parallel
```

factorial(i) =

1
2
6
24
120
720
5040

Threads access data in a disordered way

factorial(i) =

1
2

thread 0

```
factorial(1) = 1
```

```
!$omp parallel  
!$omp do schedule(static)  
do i = 2, 6
```

```
    factorial(i) = i * factorial(i-1)
```

```
end do  
!$omp end do  
!$omp end parallel
```

factorial(i) =

1
2
6
24
120
720
5040

Threads access data in a disordered way

factorial(i) =

1	
2	← thread 0
0	← thread 1

```
factorial(1) = 1
```

```
!$omp parallel  
!$omp do schedule(static)  
do i = 2, 6
```

```
    factorial(i) = i * factorial(i-1)
```

```
end do  
!$omp end do  
!$omp end parallel
```

factorial(i) =

1
2
6
24
120
720
5040

Threads access data in a disordered way

factorial(i) =

1	
2	← thread 0
0	← thread 1
0	← thread 1

```
factorial(1) = 1
```

```
!$omp parallel  
!$omp do schedule(static)  
do i = 2, 6
```

```
    factorial(i) = i * factorial(i-1)
```

```
end do  
!$omp end do  
!$omp end parallel
```

factorial(i) =

1
2
6
24
120
720
5040

Threads access data in a disordered way

factorial(i) =

1	
2	← thread 0
6	← thread 0
0	← thread 1
0	← thread 1

```
factorial(1) = 1
```

```
!$omp parallel  
!$omp do schedule(static)  
do i = 2, 6
```

```
    factorial(i) = i * factorial(i-1)
```

```
end do  
!$omp end do  
!$omp end parallel
```

factorial(i) =

1
2
6
24
120
720
5040

Threads access data in a disordered way

factorial(i) =

1	
2	← thread 0
6	← thread 0
0	← thread 1
0	← thread 1
0	← thread 2
0	← thread 2

```
factorial(1) = 1
```

```
!$omp parallel  
!$omp do schedule(static)  
do i = 2, 6
```

```
    factorial(i) = i * factorial(i-1)
```

```
end do  
!$omp end do  
!$omp end parallel
```

factorial(i) =

1
2
6
24
120
720
5040

This is called **Race Condition**

It can even be worse if factorial was not properly initialized!

Difficult to detect because for a given case, system or run, the threads may win the race in an order that happens to make the program run correctly.

Threads access data in a disordered way

factorial(i) =

1	
2	← thread 0
6	← thread 0
0	← thread 1
0	← thread 1
0	← thread 2
0	← thread 2

```
!$omp parallel
!$omp do schedule(static)
do i = 1, 6

    x = i
    y(i) = x * 2

end do
!$omp end do
!$omp end parallel
```

y(i) =

2
4
6
8
10
12

Another example of **Race Condition**

y(i) =

x	y(i)
1	

← thread 0

```
!$omp parallel
!$omp do schedule(static)
do i = 1, 6

    x = i
    y(i) = x * 2

end do
!$omp end do
!$omp end parallel
```

y(i) =

2
4
6
8
10
12

Another example of **Race Condition**

y(i) =

x	y(i)
1	2

← thread 0

```

!$omp parallel
!$omp do schedule(static)
do i = 1, 6

    x = i
    y(i) = x * 2

end do
!$omp end do
!$omp end parallel

```

y(i) =

2
4
6
8
10
12

Another example of **Race Condition**

y(i) =

x	y(i)
1	2
2	

← thread 0

← thread 0

```

!$omp parallel
!$omp do schedule(static)
do i = 1, 6

    x = i
    y(i) = x * 2

end do
!$omp end do
!$omp end parallel

```

$y(i) =$

2
4
6
8
10
12

Another example of **Race Condition**

$y(i) =$

x	y(i)	
1	2	← thread 0
2		← thread 0
5		← thread 2

```

!$omp parallel
!$omp do schedule(static)
do i = 1, 6

    x = i
    y(i) = x * 2

end do
!$omp end do
!$omp end parallel

```

y(i) =

2
4
6
8
10
12

Another example of **Race Condition**

y(i) =

x	y(i)	
1	2	← thread 0
2	10	← thread 0
5		← thread 2

```

!$omp parallel
!$omp do schedule(static)
do i = 1, 6

    x = i
    y(i) = x * 2

end do
!$omp end do
!$omp end parallel

```

$y(i) =$

2
4
6
8
10
12

Another example of **Race Condition**

$y(i) =$

x	y(i)	
1	2	← thread 0
2	10	← thread 0
3		← thread 1
5		← thread 2

```

!$omp parallel
!$omp do schedule(static)
do i = 1, 6

    x = i
    y(i) = x * 2

end do
!$omp end do
!$omp end parallel

```

$y(i) =$

2
4
6
8
10
12

Another example of **Race Condition**

$y(i) =$

x	y(i)	
1	2	← thread 0
2	10	← thread 0
3		← thread 1
5	6	← thread 2

```

!$omp parallel
!$omp do schedule(static)
do i = 1, 6

    x = i
    y(i) = x * 2

end do
!$omp end do
!$omp end parallel

```

$y(i) =$

2
4
6
8
10
12

Another example of **Race Condition**

$y(i) =$

x	y(i)	
1	2	← thread 0
2	10	← thread 0
3	6	← thread 1
5	6	← thread 2

```

!$omp parallel
!$omp do schedule(static)
do i = 1, 6

    x = i
    y(i) = x * 2

end do
!$omp end do
!$omp end parallel

```

$y(i) =$

2
4
6
8
10
12

Another example of **Race Condition**

$y(i) =$

x	y(i)	
1	2	← thread 0
2	10	← thread 0
3	6	← thread 1
4		← thread 1
5	6	← thread 2

```

!$omp parallel
!$omp do schedule(static)
do i = 1, 6

    x = i
    y(i) = x * 2

end do
!$omp end do
!$omp end parallel

```

y(i) =

2
4
6
8
10
12

Another example of **Race Condition**

y(i) =

x	y(i)	
1	2	← thread 0
2	10	← thread 0
3	6	← thread 1
4		← thread 1
5	6	← thread 2
6		← thread 2

```

!$omp parallel
!$omp do schedule(static)
do i = 1, 6

    x = i
    y(i) = x * 2

end do
!$omp end do
!$omp end parallel

```

$y(i) =$

2
4
6
8
10
12

Another example of **Race Condition**

To solve this we have the **private** clause.

$y(i) =$

x	y(i)	
1	2	← thread 0
2	10	← thread 0
3	6	← thread 1
4	12	← thread 1
5	6	← thread 2
6	12	← thread 2

```

!$omp parallel private (x)
!$omp do schedule(static)
do i = 1, 6

    x = i
    y(i) = x * 2

end do
!$omp end do
!$omp end parallel

```

Another example of **Race Condition**

To solve this we have the **private** clause.

With this, each thread has its private copy of x and the problem is solved.

This comes at the cost of an overhead in memory.

$y(i) =$

2
4
6
8
10
12

$y(i) =$

x_0	x_1	x_2	$y(i)$	
1			2	← thread 0
2			4	← thread 0
	3		6	← thread 1
	4		8	← thread 1
		5	10	← thread 2
		6	12	← thread 2

Can you declare all private variables of the following code?

NOTE: jumpx and w1d are outputs of the corresponding subroutine calls.

```
!$omp parallel private (???)
!$omp do schedule(static)
do i = ini, end
  ii = 1+dim*(i-1)
  do k = 1, nvi(i)
    j = neighbors(i,k)
    call apply_PBC(i,k,0,jumpx)
    d1 = a(ii) - a(jj) - jumpx
    d05 = sqrt(d1)
    v1 = d05/h(i)
    call Wkernel(v1,w1d)
    dter = pk(i)*w1d
    sumwh(i) = sumwh(i) + xmass(j) * dter
    if (equ) then
      dlw1d = log10(w1d)/indice(i)
    end if
  end do
do it = 1, nut
  do ie = 1, ne
    f(ie, it) = kfactor * alpha( ie, it, i)
    ftot(ie, it, i) = f(ie, it) / dens(i)
  end do
end do
end do
!$omp end do
!$omp end parallel
```

A note regarding private variables: **They are not initialized!**

```
result = 20

!$omp parallel private(result)
result = result + 10
print *, 'Thread ', omp_get_thread_num(), result
!$omp end parallel

print *, result
```

To control the initialization of private variables we have **firstprivate**

```
Thread 3 10
Thread 0 10
Thread 1 71
Thread 2 10
```

```
20
```

If you are lucky, result may access a section of the memory that is empty. Otherwise, any value can be used to initialize result.

A note regarding private variables: **They are not initialized!**

```
result = 20

!$omp parallel firstprivate(result)
result = result + 10
print *, 'Thread ', omp_get_thread_num(), result
!$omp end parallel

print *, result
```

To control the initialization of private variables we have **firstprivate**

```
Thread 3 30
Thread 0 30
Thread 1 30
Thread 2 30

20
```

Note that this output has no relation with the OpenMP section! Private variables are destroyed once the parallel section ends.

Consider now this code. Is it correct?

```
sum = 0

!$omp parallel
!$omp do
do i = 1, 6

    sum = sum + i

end do
!$omp end do
!$omp end parallel
```

The variable `sum` should be private to give the correct answer, but it should be also shared to be accessed by all threads!

Consider now this code. Is it correct?

```
sum = 0

!$omp parallel
!$omp do reduction(+: sum)
do i = 1, 6

    sum = sum + i

end do
!$omp end do
!$omp end parallel
```

The variable sum should be private to give the correct answer, but it should be also shared to be accessed by all threads!

We can solve this with **reduction**

This allows each thread to have a private copy of sum where they store their partial calculation and then, when the threads exit, it groups all partial values from all threads using the defined operation (+ in this case) in one global variable.

Reduction variables have to meet the following requirements:

- They can only be listed in one reduction
- Cannot be declared constant
- Cannot be declared private in the parallel construct

Consider now this code. Is it correct?

```
sum = 0
```

```
!$omp parallel
```

```
!$omp do reduction(+: sum)
```

```
do i = 1, 6
```

```
    sum = sum
```

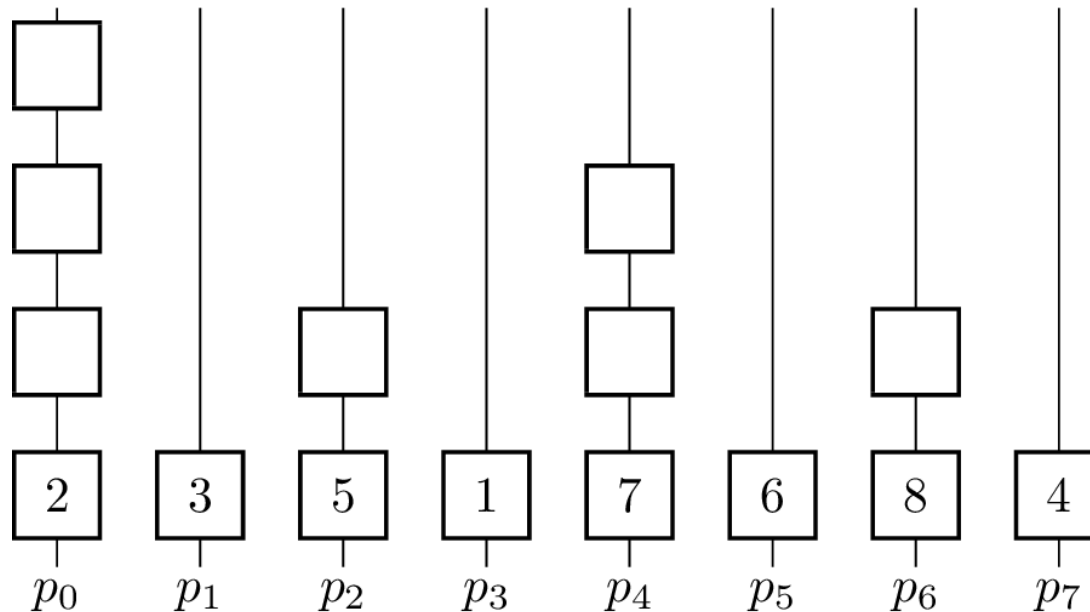
```
end do
```

```
!$omp end do
```

```
!$omp end par
```

This allows each
they store their p
exit, it groups all p
operation (+ in thi

The variable sum should be private to give the correct answer, but it should be also shared to be accessed by



Source: wikipedia

- They can only be listed in one reduction
- Cannot be declared constant
- Cannot be declared private in the parallel construct

Getting the course material

Clone the repository of the material:

```
git clone https://git.scicore.unibas.ch/scicore/openmp_course.git
```

Or download them directly and extract:

https://git.scicore.unibas.ch/scicore/openmp_course/-/archive/master/openmp_course-master.tar.gz

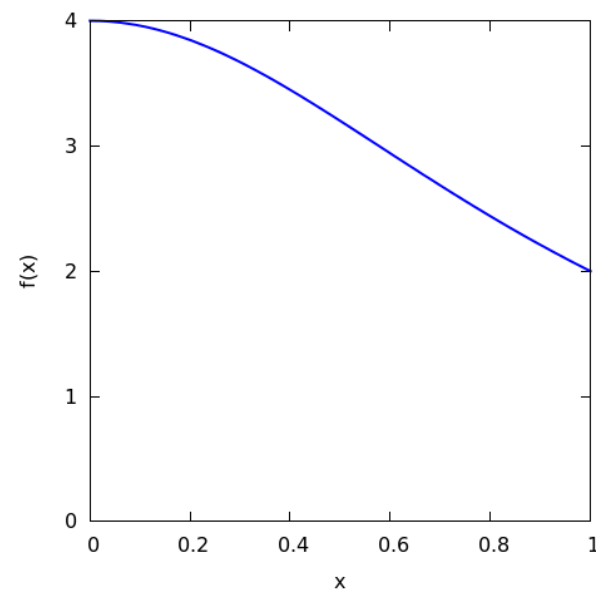
Enter in openmp_course/:

```
cd openmp_course
```

Exercise: Numerical integration

We know that:

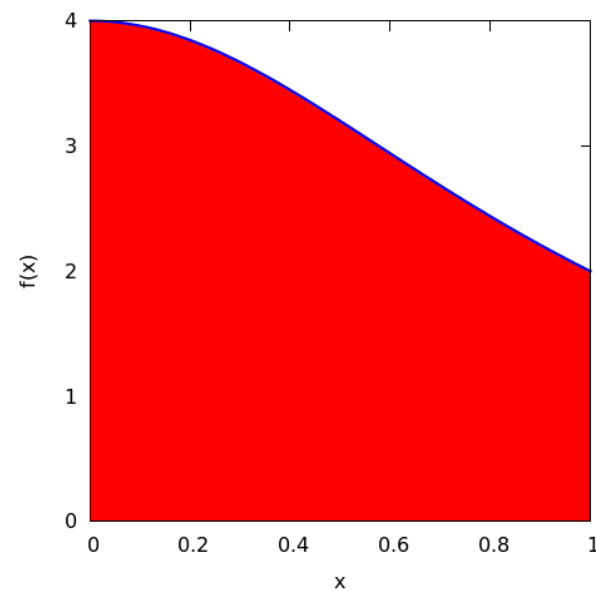
$$\int_0^1 \frac{4}{(1+x^2)} dx = \pi$$



Exercise: Numerical integration

We know that:

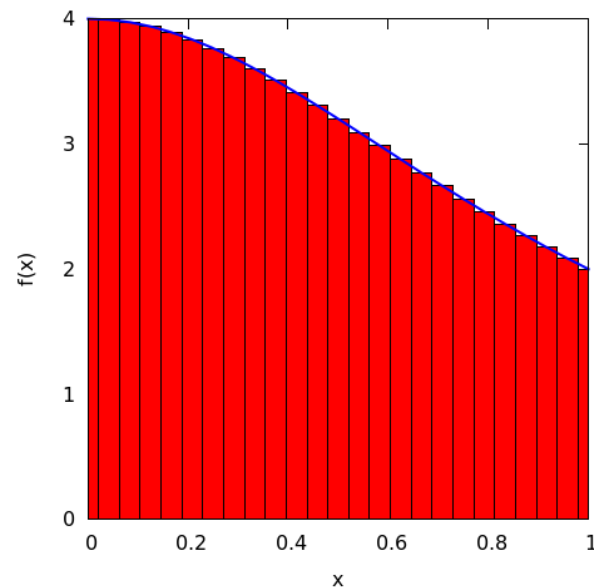
$$\int_0^1 \frac{4}{(1+x^2)} dx = \pi$$



Exercise: Numerical integration

We know that:

$$\int_0^1 \frac{4}{(1+x^2)} dx = \pi$$



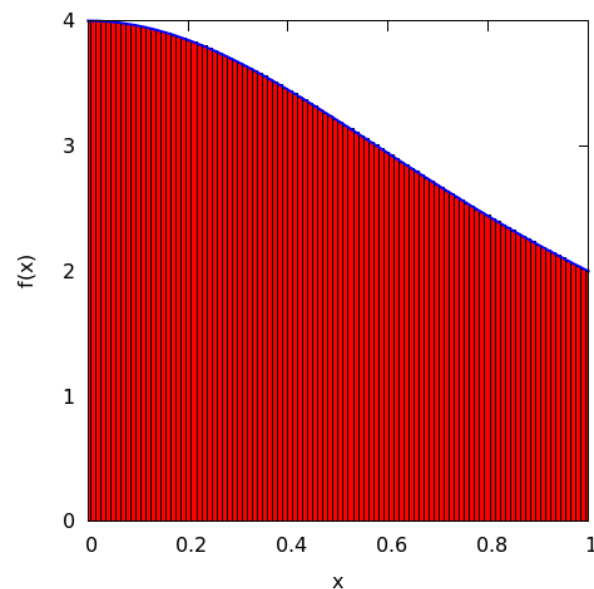
Using the composite formula we can approximate the integral as:

$$\int_a^b f(x) dx \approx \left(\sum_{i=0}^n f(x_i) \right) \frac{b-a}{n}$$

Exercise: Numerical integration

We know that:

$$\int_0^1 \frac{4}{(1+x^2)} dx = \pi$$



Using the composite formula we can approximate the integral as:

$$\int_a^b f(x) dx \approx \left(\sum_{i=0}^n f(x_i) \right) \frac{b-a}{n}$$

Exercise: Numerical integration

Parallelize the following program with openMP

```
steps = 1000000000
dx = 1./dble(steps)
sum = 0.
do i = 1, steps
    x = (dble(i)-0.5)*dx
    sum = sum + 4./(1. + x*x)
end do
sum = sum * dx
```

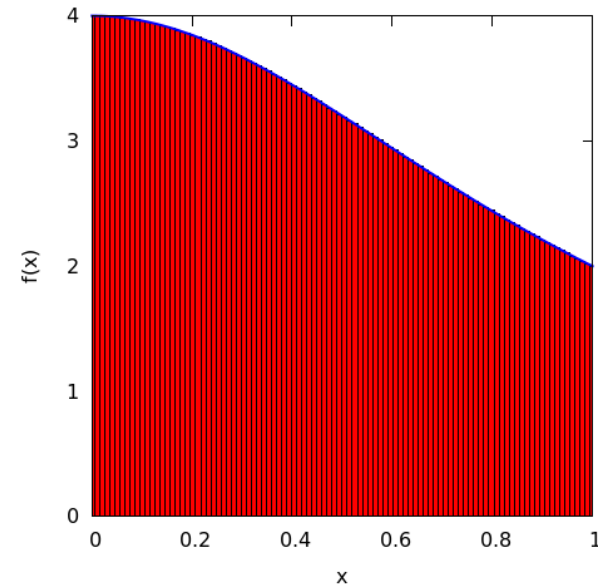
Compile with: **gfortran -fopenmp pi.f90**

Fix the number of threads: **export OMP_NUM_THREADS=4**

Execute with: **time ./a.out**

We know that:

$$\int_0^1 \frac{4}{(1+x^2)} dx = \pi$$



Using the composite formula we can approximate the integral as:

$$\int_a^b f(x) dx \approx \left(\sum_{i=0}^n f(x_i) \right) \frac{b-a}{n}$$

What happens if we have nested loops?

```
do i = 1, ni
  do j = 1, nj
    do k = 1, nk
      <calculations>
    end do
  end do
end do
```

What happens if we have nested loops?

Not much... we simply parallelize the outer loop.

```
!$omp parallel private(i,j,k)
!$omp do
do i = 1, ni
  do j = 1, nj
    do k = 1, nk
      <calculations>
    end do
  end do
end do
!$omp end do
!$omp end parallel
```

But this depends on the number of iterations for each loop.

What happens if we have nested loops?

Not much... we simply parallelize the outer loop.

```
!$omp parallel private(i,j,k)
!$omp do
do i = 1, 4
  do j = 1, 20
    do k = 1, 10000
      <calculations>
    end do
  end do
end do
!$omp end do
!$omp end parallel
```



But this depends on the number of iterations for each loop.

Having such small count of iterations in the outer loop is not efficient. We might even have less iterations than threads!

What happens if we have nested loops?

Not much... we simply parallelize the outer loop.

```
!$omp parallel private(i,j,k)
!$omp do
do k = 1, 10000
  do j = 1, 20
    do i = 1, 4
      <calculations>
    end do
  end do
end do
!$omp end do
!$omp end parallel
```

But this depends on the number of iterations for each loop.

Having such small count of iterations in the outer loop is not efficient. We might even have less iterations than threads!

Re-arranging the nested loops will solve it!

But this is not always possible, or maybe all loops have relatively small counts!

What happens if we have nested loops?

Not much... we simply parallelize the outer loop.

```
!$omp parallel private(i,j,k)
!$omp do
do i = 1, 10
  do j = 1, 10
    do k = 1, 10
      <calculations>
    end do
  end do
end do
!$omp end do
!$omp end parallel
```

But this depends on the number of iterations for each loop.

Having such small count of iterations in the outer loop is not efficient. We might even have less iterations than threads!

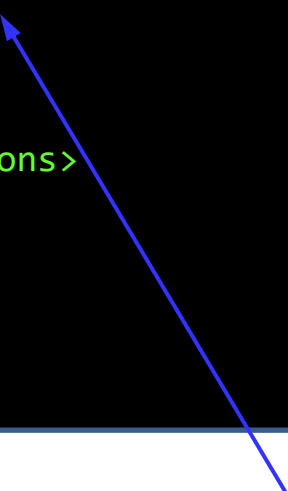
Re-arranging the nested loops will solve it!

But this is not always possible, or maybe all loops have relatively small counts!

What happens if we have nested loops?

Not much... we simply parallelize the outer loop.

```
!$omp parallel private(i,j,k)
!$omp do collapse(3)
do i = 1, 10
  do j = 1, 10
    do k = 1, 10
      <calculations>
    end do
  end do
end do
!$omp end do
!$omp end parallel
```



To solve this we have the clause **collapse(n)**

It specifies how many nested loops will be collapsed in a single loop.

But this depends on the number of iterations for each loop.

Having such small count of iterations in the outer loop is not efficient. We might even have less iterations than threads!

Re-arranging the nested loops will solve it!

But this is not always possible, or maybe all loops have relatively small counts!



```
!$omp parallel private(ii,i,j,k)
!$omp do
do ii = 1, 1000
  i = mod(ii/100, 10)+1
  j = mod(ii/10, 10)+1
  k = mod(ii, 10)+1
  <calculations>
end do
!$omp end do
!$omp end parallel
```

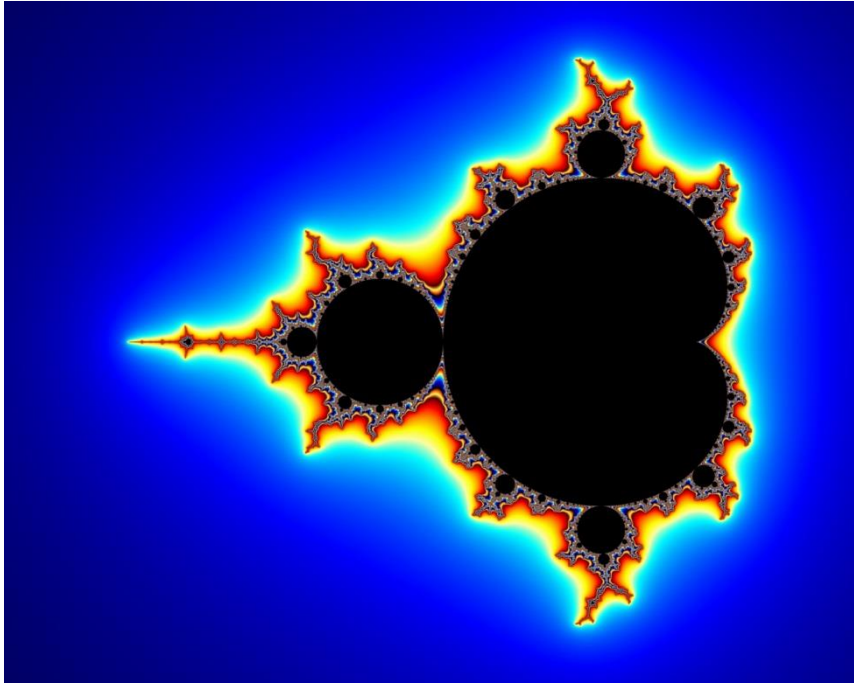
What about subroutines? We can use **orphaned directives**

```
!$omp parallel private(i,j,k)  
call calculate_manythings()  
!$omp end parallel
```



```
subroutine calculate_manythings()  
  
!$omp do schedule(static)  
  do i=1,n  
    <calculations>  
  enddo  
!$omp end do  
  
return  
end subroutine
```

Mandelbrot set area calculation



Mathematically it can be calculated exactly as:

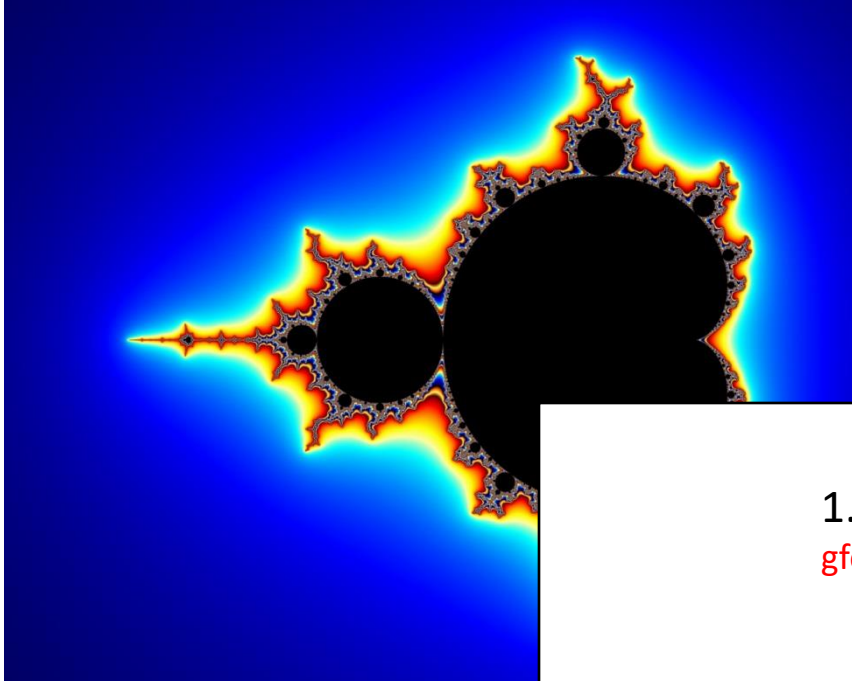
$$A = \pi \left(1 - \sum_{n=1}^{\infty} n b_n^2 \right)$$

But this series converges VERY slowly. It needs 10^{118} terms to get the first two digits, and 10^{1181} to get the third!

The area obtained by pixel counting is:

1.50659177 ± 0.00000008
(Munafo et al. OEIS A098403, 2000)

Mandelbrot set area calculation



Mathematically it can be calculated exactly as:

$$A = \pi \left(1 - \sum_{n=1}^{\infty} n b_n^2 \right)$$

But this series converges VERY slowly. It needs 10^{118} terms to get the first two digits, and 10^{1181} to get the third!

The area obtained by pixel counting is:

1.50659177 ± 0.00000008

(Munafo et al. OEIS A098403, 2000)

1. Compile and execute the serial code

```
gfortran mandelbrot.f90 -o mandelbrot_serial  
time ./mandelbrot_serial
```

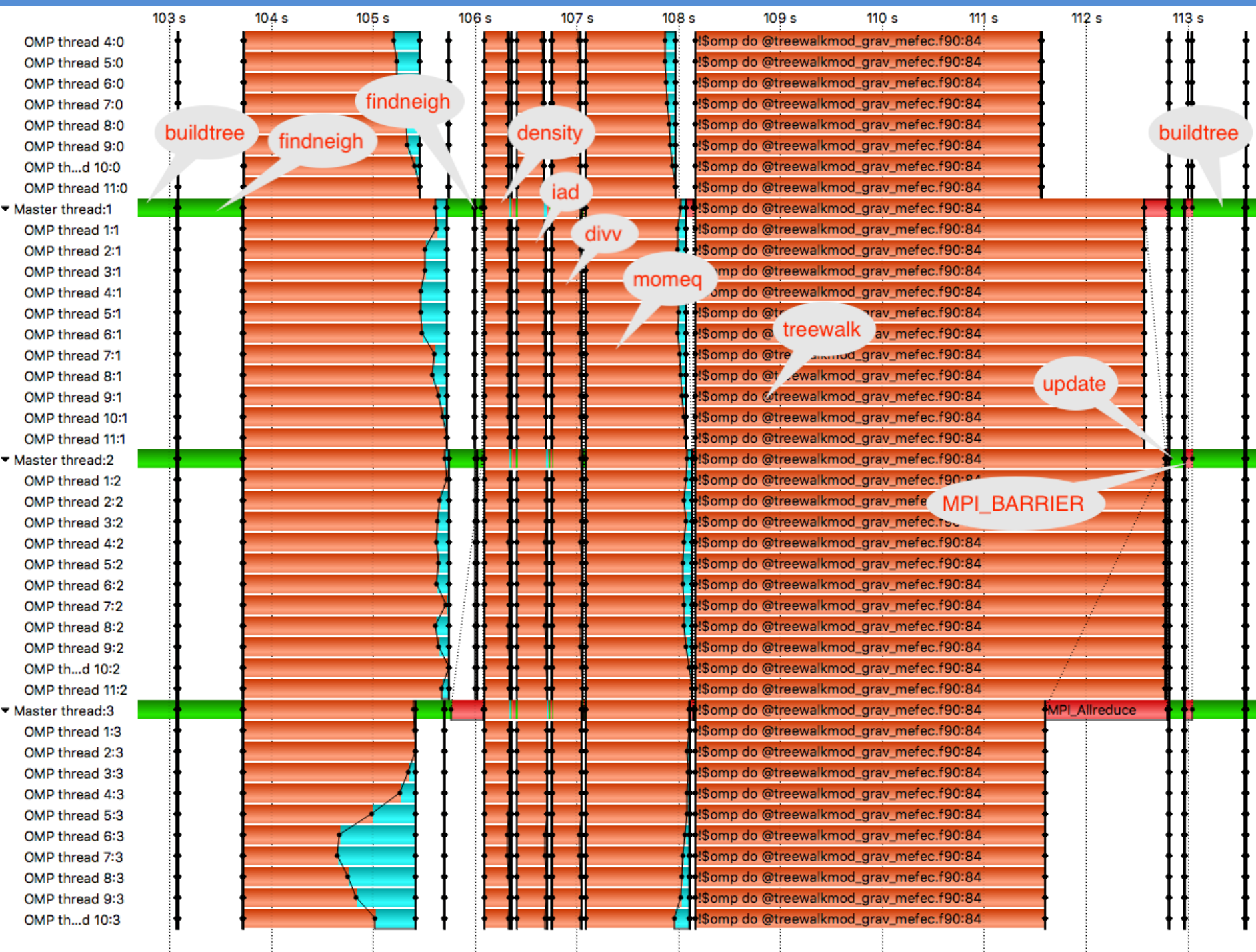
2. Parallelize the code with OpenMP

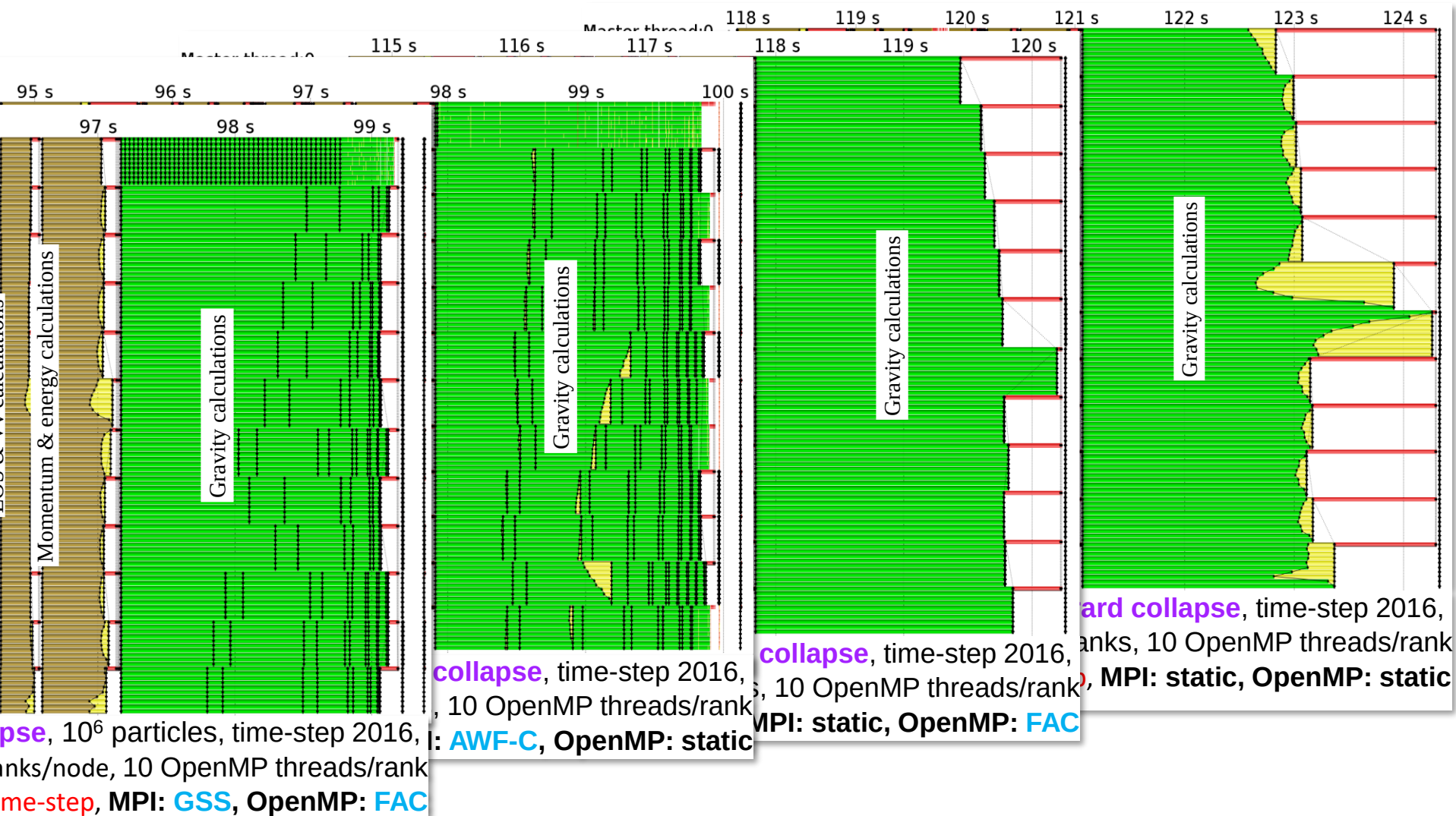
3. Compile and execute the parallel code

```
gfortran -fopenmp mandelbrot.f90 -o mandelbrot_parallel  
time ./mandelbrot_parallel
```

4. Change the number of threads (2-4) and check if it scales

```
export OMP_NUM_THREADS = <# of threads>
```





Can we do any of this in Python? **Kind of...**

```
from multiprocessing import Process

def func1():
    j=0
    print ('func1: starting')
    for i in range(1000000000):
        j=j+1
    print ('func1: finishing',j)

def func2():
    j=0
    print ('func2: starting')
    for i in range(1000000000):
        j=j+1
    print ('func2: finishing',j)

def func3():
    j=0
    print ('func3: starting')
    for i in range(1000000000):
        j=j+1
    print ('func3: finishing',j)

if __name__ == '__main__':
    p1 = Process(target=func1)
    p1.start()
    p2 = Process(target=func2)
    p2.start()
    p3 = Process(target=func3)
    p3.start()
    p1.join()
    p2.join()
    p3.join()
```

```
top - 11:28:00 up 2 days, 2:02, 1 user, load average: 0,57, 0,56, 0,48
Tasks: 306 total, 4 running, 301 sleeping, 0 stopped, 1 zombie
%Cpu(s): 38,3 us, 0,4 sy, 0,0 ni, 61,1 id, 0,1 wa, 0,0 hi, 0,1 si, 0,0 st
KiB Mem : 8079052 total, 294168 free, 5072908 used, 2711976 buff/cache
KiB Swap: 19529724 total, 19506740 free, 22984 used. 1438376 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
26027	ruben	20	0	50896	8488	3072	R	100,0	0,1	0:09.43	python
26028	ruben	20	0	50896	8488	3072	R	100,0	0,1	0:09.43	python
26026	ruben	20	0	50896	8488	3072	R	99,0	0,1	0:09.43	python
24587	ruben	20	0	4736720	2,196g	2,119g	S	4,0	28,5	7:45.81	VirtualBox
1128	root	20	0	756332	231528	197116	S	1,0	2,9	11:15.62	Xorg
7796	ruben	20	0	1325324	184588	52036	S	1,0	2,3	21:43.70	skypeforli+
26032	ruben	20	0	437152	21932	18568	S	1,0	0,3	0:00.25	gnome-scre+
1	root	20	0	185332	4984	3008	S	0,0	0,1	0:01.58	systemd

```
ruben@jarvis:~/test/parallepython$ time python test.py
func1: starting
func2: starting
func3: starting
func3: finishing 1000000000
func2: finishing 1000000000
func1: finishing 1000000000
[1]+  Done                  emacs test.py

real    0m36.208s
user    1m50.284s
sys     0m0.544s
```

Can we do any of this in Python? **Kind of...**

```
from multiprocessing import Process

def func1():
    j=0
    print ('func1: starting')
    for i in range(1000000000):
        j=j+1
    print ('func1: finishing',j)

def func2():
    j=0
    print ('func2: starting')
    for i in range(1000000000):
        j=j+1
    print ('func2: finishing',j)

def func3():
    j=0
    print ('func3: starting')
    for i in range(1000000000):
        j=j+1
    print ('func3: finishing',j)

def runInParallel(*fns):
    proc = []
    for fn in fns:
        p = Process(target=fn)
        p.start()
        proc.append(p)
    for p in proc:
        p.join()

if __name__ == '__main__':
    runInParallel (func1, func2, func3)
```

```
top - 11:28:00 up 2 days, 2:02, 1 user, load average: 0,57, 0,56, 0,48
Tasks: 306 total, 4 running, 301 sleeping, 0 stopped, 1 zombie
%Cpu(s): 38,3 us, 0,4 sy, 0,0 ni, 61,1 id, 0,1 wa, 0,0 hi, 0,1 si, 0,0 st
KiB Mem : 8079052 total, 294168 free, 5072908 used, 2711976 buff/cache
KiB Swap: 19529724 total, 19506740 free, 22984 used. 1438376 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
26027	ruben	20	0	50896	8488	3072	R	100,0	0,1	0:09.43	python
26028	ruben	20	0	50896	8488	3072	R	100,0	0,1	0:09.43	python
26026	ruben	20	0	50896	8488	3072	R	99,0	0,1	0:09.43	python
24587	ruben	20	0	4736720	2,196g	2,119g	S	4,0	28,5	7:45.81	VirtualBox
1128	root	20	0	756332	231528	197116	S	1,0	2,9	11:15.62	Xorg
7796	ruben	20	0	1325324	184588	52036	S	1,0	2,3	21:43.70	skypeforli+
26032	ruben	20	0	437152	21932	18568	S	1,0	0,3	0:00.25	gnome-scre+
1	root	20	0	185332	4984	3008	S	0,0	0,1	0:01.58	systemd

```
ruben@jarvis:~/test/parallepython$ time python test.py
func1: starting
func2: starting
func3: starting
func3: finishing 1000000000
func2: finishing 1000000000
func1: finishing 1000000000
[1]+  Done                  emacs test.py

real    0m36.208s
user    1m50.284s
sys     0m0.544s
```

<http://stackabuse.com/parallel-processing-in-python/>

We want to sum up the integer half-value of the first $3e8$ integers.

```
from multiprocessing import Process, Manager

def func1(id, results):
    j=0
    print('func1: starting')
    for i in range(100000000):
        j=j+i/2
    print('func1: finishing')
    results[id]=j

def func2(id, results):
    j=0
    print('func2: starting')
    for i in range(100000000, 200000000):
        j=j+i/2
    print('func2: finishing')
    results[id]=j

def func3(id, results):
    j=0
    print('func3: starting')
    for i in range(200000000, 300000000):
        j=j+i/2
    print('func3: finishing')
    results[id]=j

def runInParallel(*fns):
    proc=[]
    for i, fn in enumerate(fns):
        p = Process(target=fn, args=(i, results))
        p.start()
        proc.append(p)
    for p in proc:
        p.join()
    print(results)
    print(sum(results.values()))

if __name__ == '__main__':
    results=Manager().dict()
    runInParallel (func1, func2, func3)
```

```
[cabezon@login10 openmp]$ time python python_multiproc_2.py
func1: starting
func2: starting
func3: starting
func3: finishing
func2: finishing
func1: finishing
{0: 2499999950000000, 1: 7499999900000000, 2: 12499999850000000}
22499999700000000

real    0m12.819s
user    0m27.136s
sys     0m8.865s
[cabezon@login10 openmp]$
```

Note that threads finish in an unstructured way!

We store the results in a dictionary and sum them up.
(mimicking a 'reduce' in OpenMP)

More elegant way to do this is using 'numba'. <https://numba.pydata.org/>



Adding a decorator numba compiles the code on-the-fly creating an optimized machine code

```
from numba import jit

@jit()
def func1():
    j=0
    print('func1: starting')
    for i in range(300000000):
        j=j+i/2
    print('func1: finishing',j)

if __name__ == '__main__':
    func1()
```

```
[cabezon@login10 openmp]$ time python python_multiproc_2.py
func1: starting
func2: starting
func3: starting
func3: finishing
func2: finishing
func1: finishing
{0: 24999999500000000, 1: 74999999000000000, 2: 124999998500000000}
224999997000000000

real    0m12.819s
user    0m27.130s
sys     0m8.865s
[cabezon@login10 openmp]$
```

```
[cabezon@login10 openmp]$ source numbatest/bin/activate
(numbatest) [cabezon@login10 openmp]$ time python python_numba.py
func1: starting
func1: finishing 2.249999988355443e+16

real    0m1.142s
user    0m1.205s
sys     0m1.294s
(numbatest) [cabezon@login10 openmp]$
```

More elegant way to do this is using 'numba'. <https://numba.pydata.org/>



Adding a decorator numba compiles the code on-the-fly creating an optimized machine code

```
from numba import jit
@jit()
def func1():
    j=0
    print('func1: starting')
    for i in range(3000000):
        j=j+i/2
    print('func1: finishing')

if __name__ == '__main__':
    func1()
```

```
[cabezon@login10 openmp]$ time python python_multiproc_2.py
func1: starting
func2: starting
func3: starting
func3: finishing
func2: finishing
func1: finishing
{0: 2499999950000000, 1: 7499999900000000, 2: 12499999850000000}
22499999700000000

real    0m12.819s
user    0m27.130s
sys     0m8.865s
```

We need to install numba. For that we will install a virtual environment of Python.

(We'll use **uv** as 21st century researchers...)

```
uv venv
uv pip install numba
source .venv/bin/activate
```

3. Execute the serial version with the numba decorator

(don't forget to import jit at the beginning of the Python script)

```
time python python_serial.py
```

More elegant way to do this is using 'numba'. <https://numba.pydata.org/>



We are using just 1 core with numba, but we can also use all cores in the machine, just like with OpenMP, with prange:

```
cabezon@shil01:~/openmp_course $time python python_serial.py
func: starting (from 1 to 30000000000)
func1: finishing 2.2500000000007956e+20

real    0m31.206s
user    0m32.340s
sys     0m2.832s
cabezon@shil01:~/openmp_course $
```

```
cabezon@shil01:~/openmp_course $time python python_serial.py
func: starting (from 1 to 30000000000)
func1: finishing 2.2500000000005246e+20

real    0m1.781s
user    0m41.160s
sys     0m3.124s
cabezon@shil01:~/openmp_course $
```

```
from numba import jit,prange

@jit(parallel=True)
def func1():
    j=0
    print('func: starting (from 1 to 3000000000)')
    for i in prange(1,30000000001):
        j=j+1/2
    print('func1: finishing',j)

if __name__ == '__main__':
    func1()
```

NOTE: we are doing x100 more iterations in this example than in the previous one so that we can actually see differences in the times between 1 and many cores.

```
saktho00@worker04:~/temp/numbatest
File Edit View Search Terminal Help

    genotypes[cellID,2+targetoffset]=maxsnps#genotypes[c
ellID,3+targetoffset] + 1
    #genotypes[cellThreadID,1+targetoffset]=gen
    #genotypes[cellID,2+targetoffset]=genotypes[cellID,2+targetoffse
t])+ 1
    cuda.syncthreads()
    #cuda.atomic.add(syncObj,0,1)
    #while(syncObj[0]<N*(numbIt+1+gen)):
    #    1+1
@jit
def pairwisedist(genotypes):
    difflist = []
    N=int(genotypes.shape[0])
    L=int(genotypes.shape[1]/2)
    for i in range(N-1):
        for j in range(i+1,N):
            delta=0
            diff = 0
            for k in range(L):
                delta = genotypes[i,k] ^ genotypes[j,k]
                for m in range(64):
                    if int64(delta) & 1 == 1:
                        diff = diff + 1
                delta = math.floor(delta / 2)
            if delta == 0:
                break
            difflist.append(diff/(L*64))
    return(difflist)
@jit
def closedist(genotypes):
    "recombSim.py" 329L, 11141C written
```

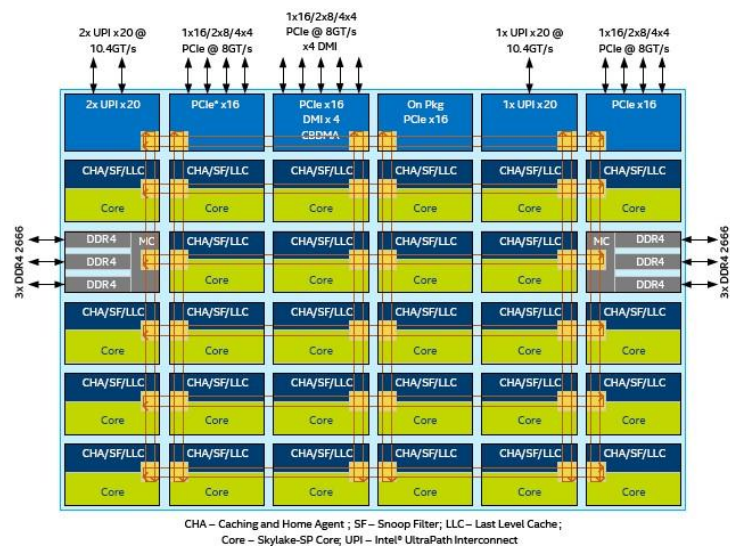
Thomas Sakoparnig

```
saktho00@worker04:~/temp/numbatest
File Edit View Search Terminal Help

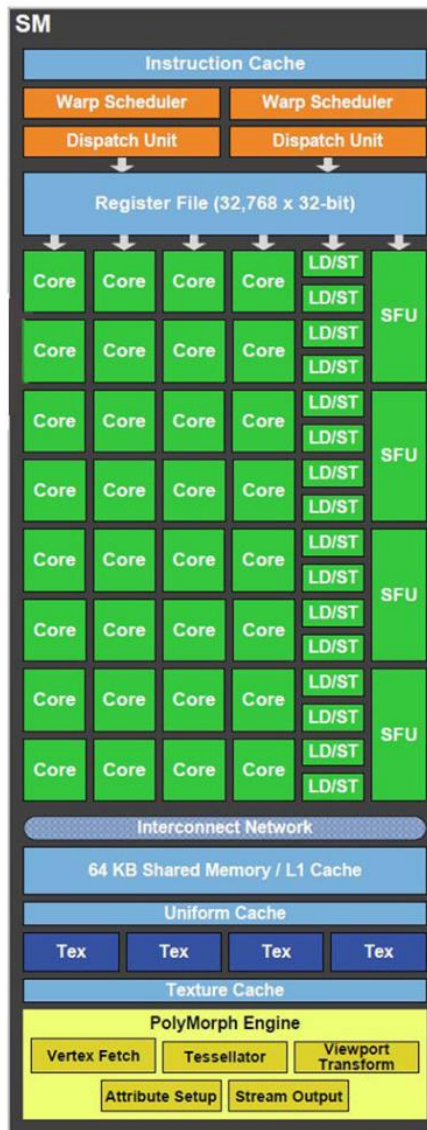
[saktho00@sgi27 numbatest]$ vi recombSim.py
[saktho00@sgi27 numbatest]$ python recombSim.py
ttime: 3.4206590335816145
0.0170744024235
[saktho00@sgi27 numbatest]$ vi recombSim.py
[saktho00@sgi27 numbatest]$ python recombSim.py
ttime: 2637.8457502331585
0.0178537920918
[saktho00@sgi27 numbatest]$
```

What about GPU?

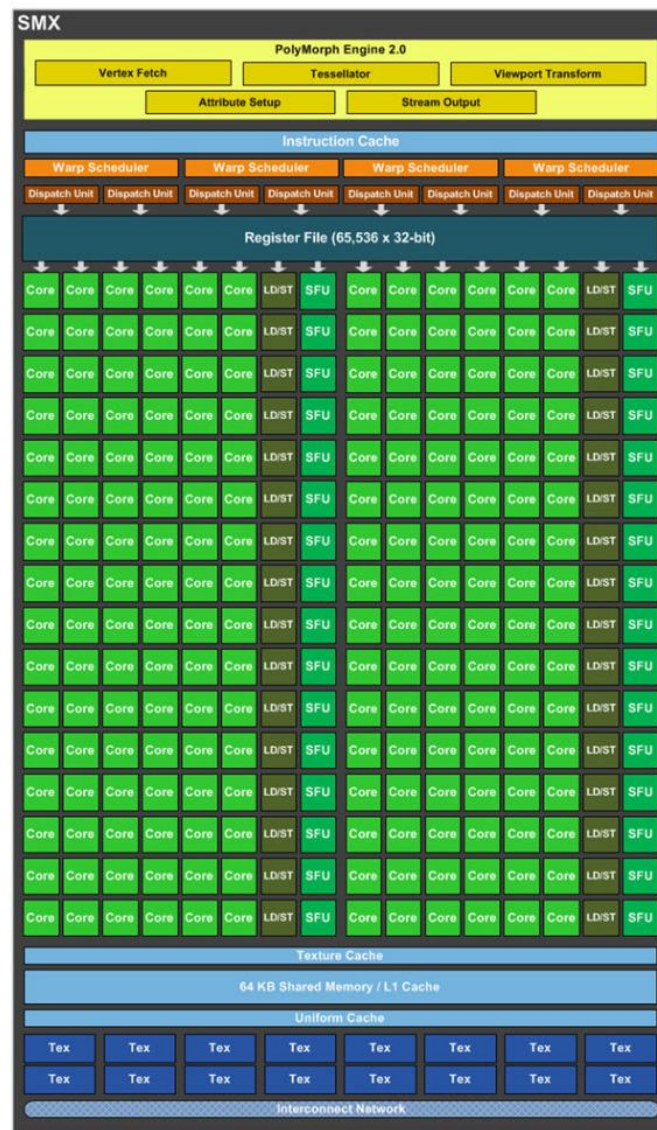
Intel Skylake (2017)



Tesla SM unit (2007)



Fermi SMX unit (2011)

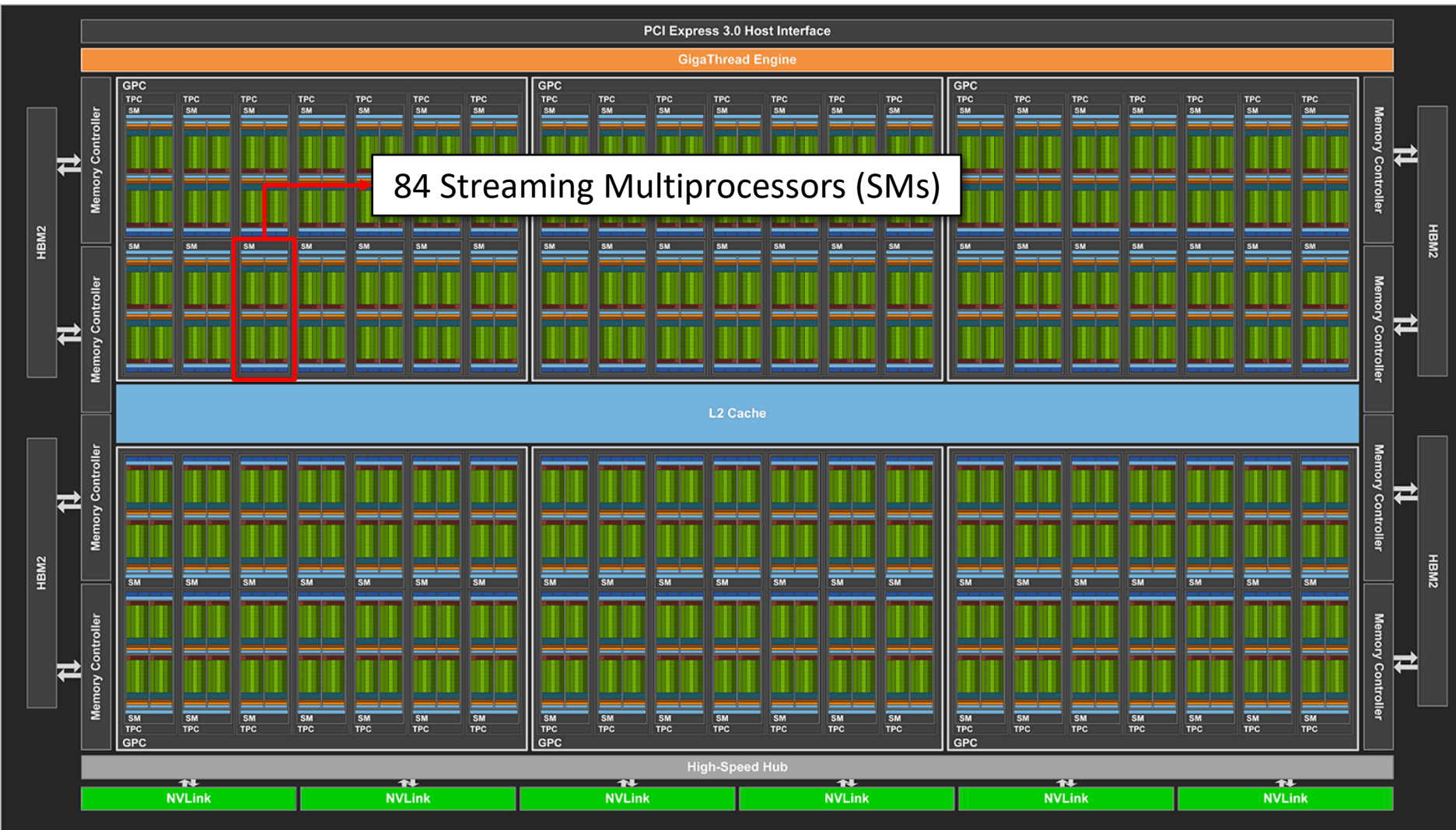


Volta (2017)

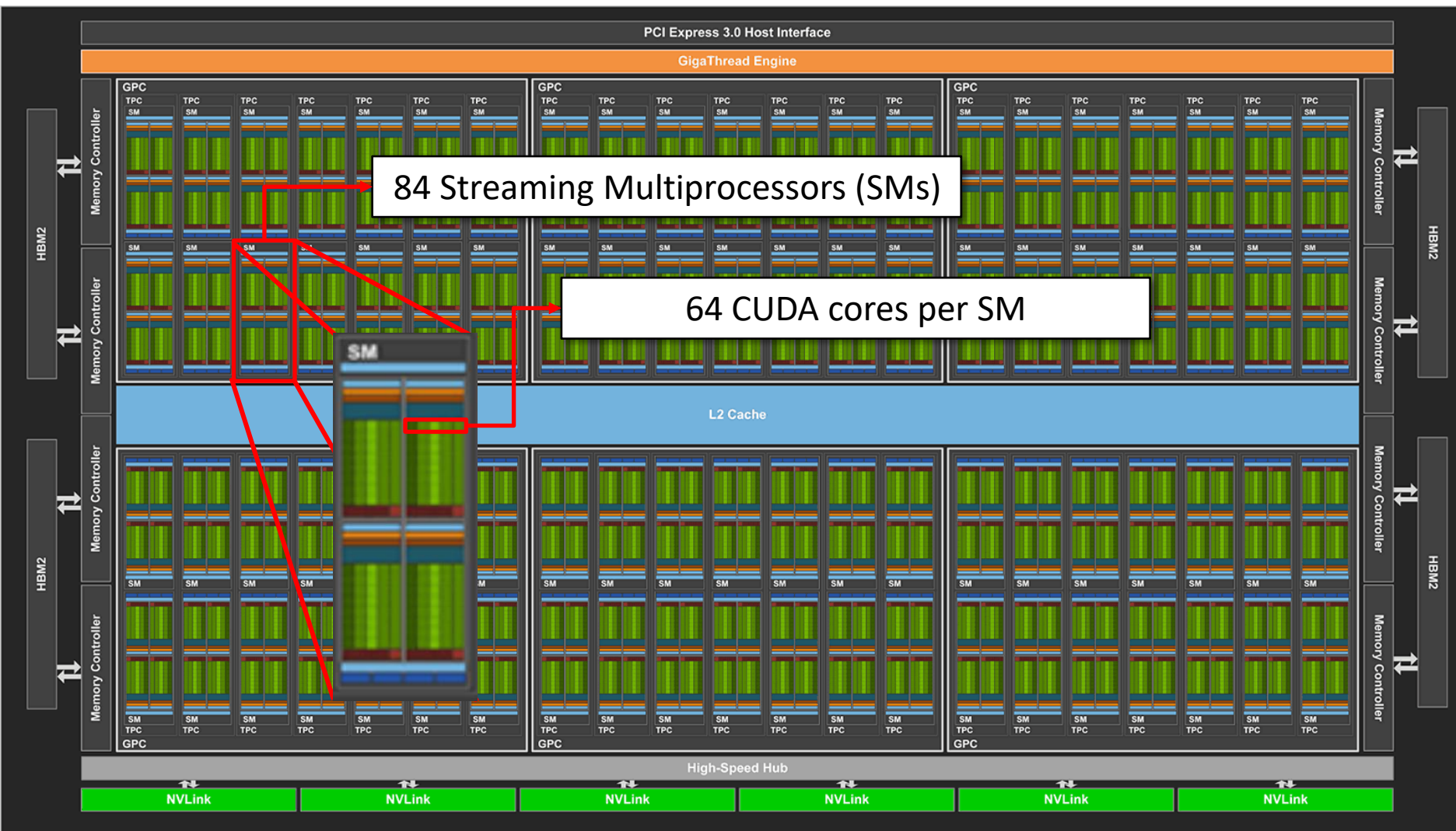


Volta (2017)

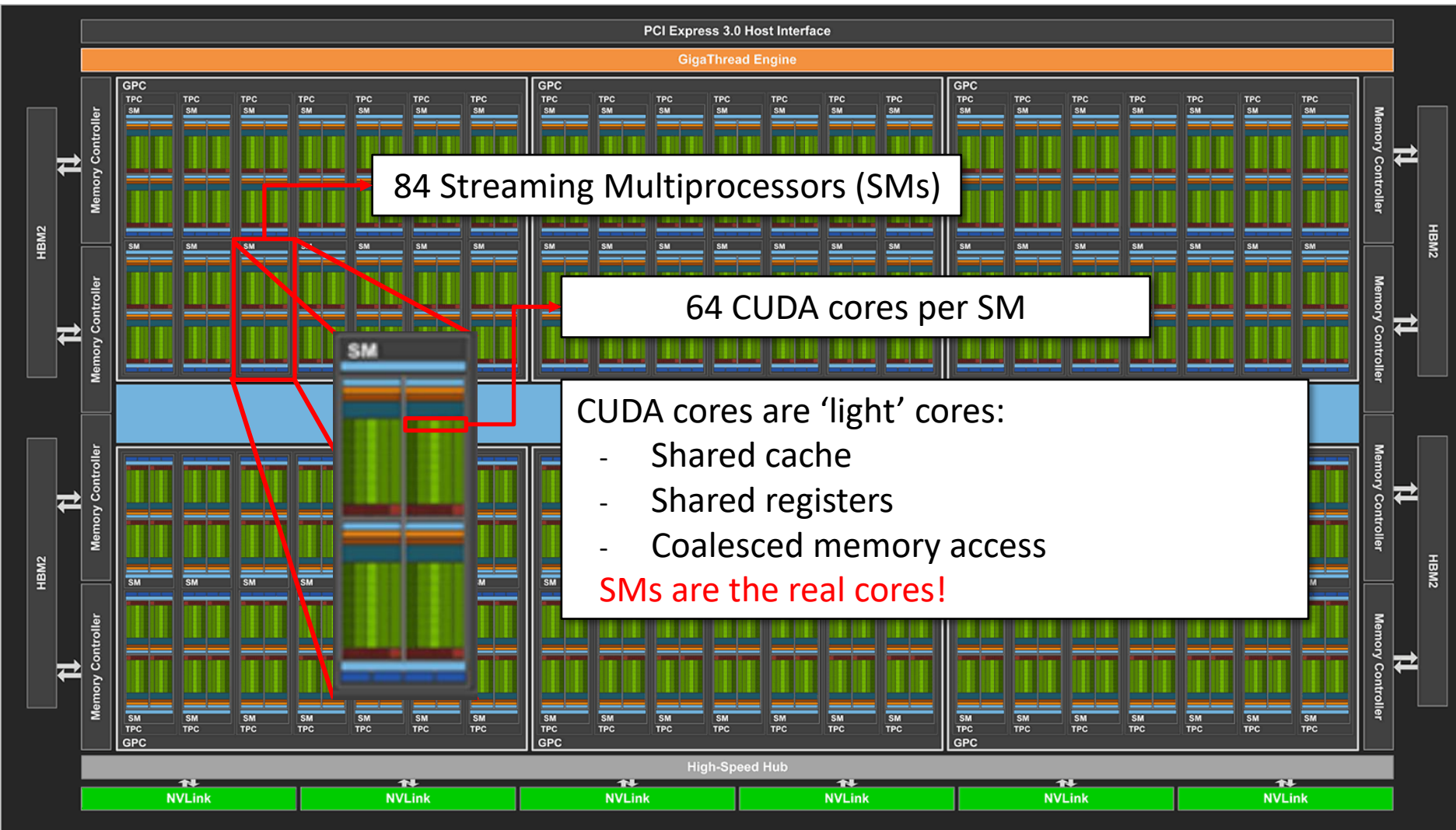
84 Streaming Multiprocessors (SMs)



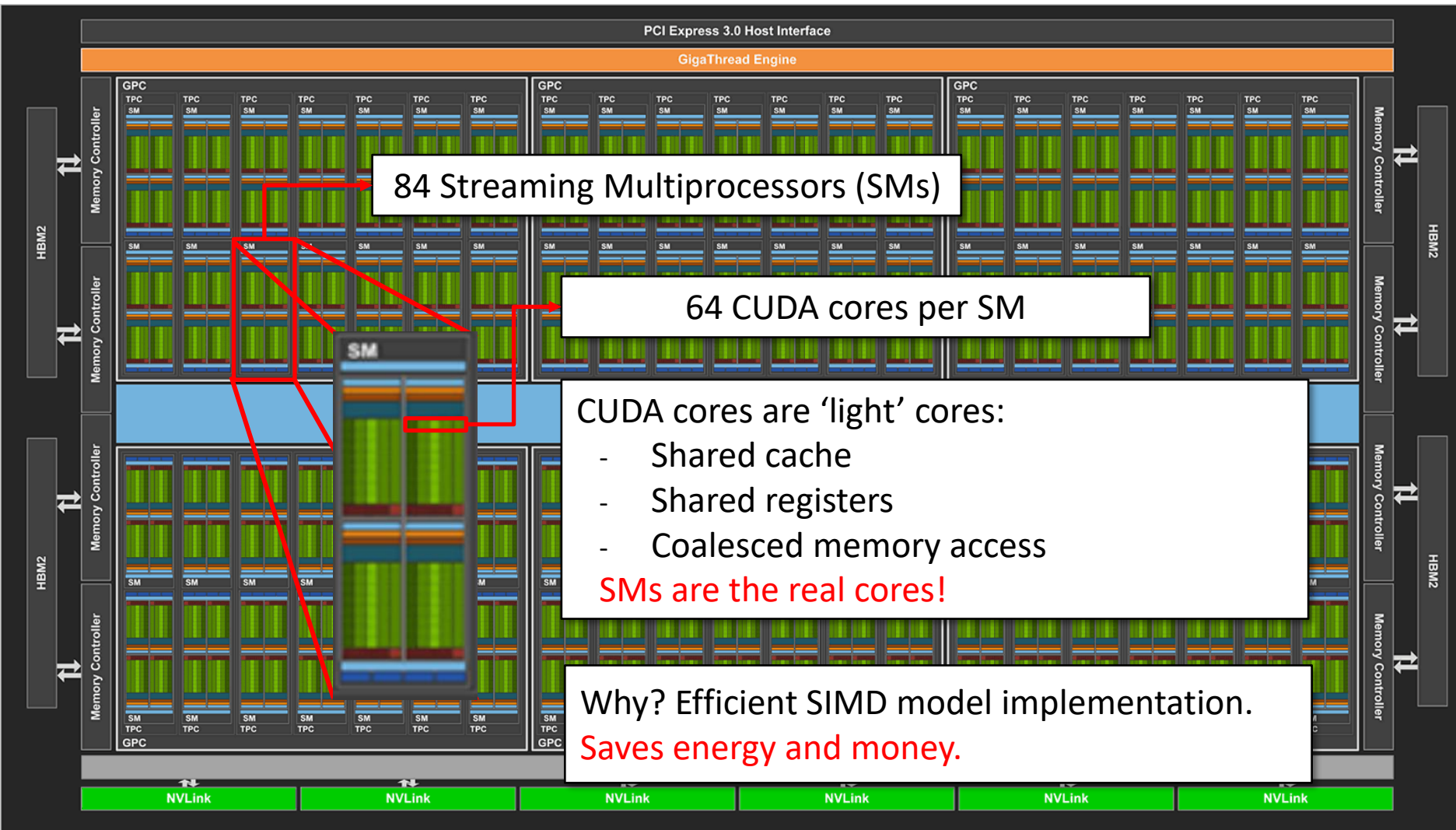
Volta (2017)



Volta (2017)



Volta (2017)



Since version 4.0, openMP can do GPU offloading.

```
int A[1] = {-1};
#pragma omp target
{
    A[0] = omp_is_initial_device();
}
if (!A[0]) {
    printf("Able to use offloading!\n");
}
```

`omp_is_initial_device()` returns 0 when called from the accelerator

You can control the transfer of data

```
#pragma omp target data map(to: b[:n])
{
    // 'b' copied onto device

    // This region is offloaded to device
    #pragma omp target map(tofrom: a[:n])
    {
        // 'a' copied onto device
        #pragma omp teams distribute
        for(int ii = 0; ii < n; ++ii)
        {
            a[ii] = a[ii] + alpha * b[ii];
        }
    }
    // 'a' copied back

    // ... potentially more target regions
}
// 'b' is discarded by device
```

Since version 4.0, openMP can do **GPU offloading**.

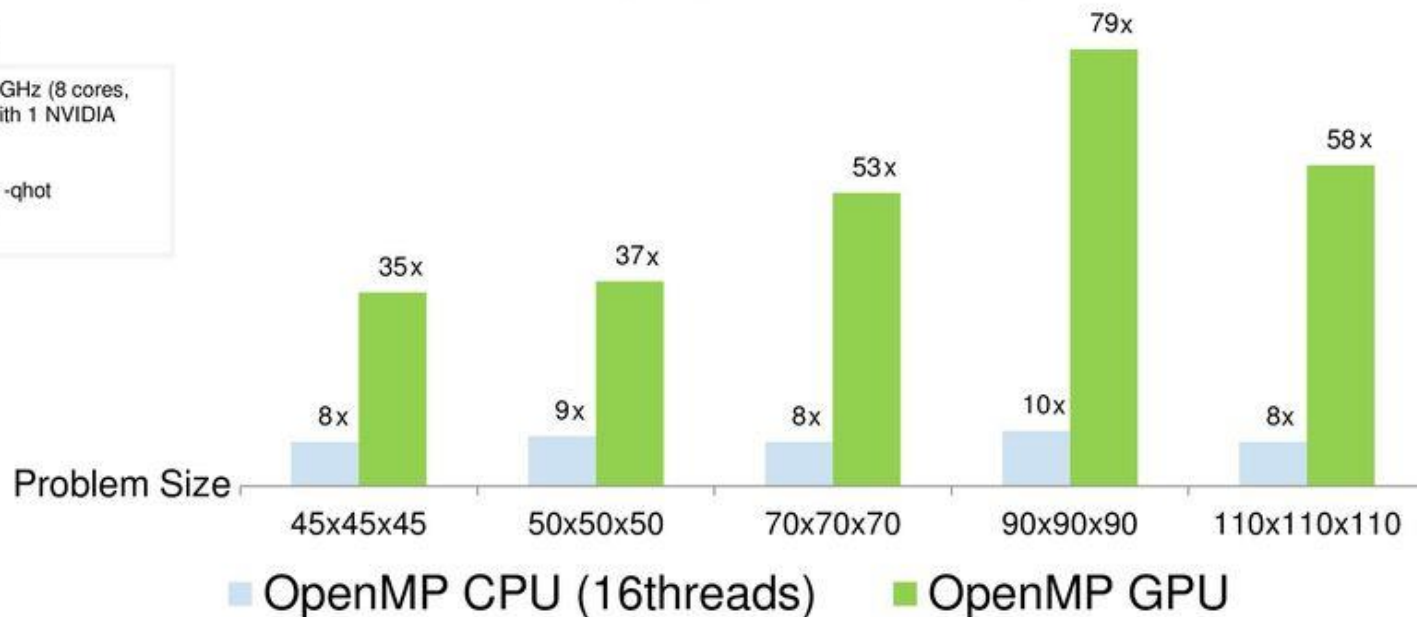
Performance Results – End to End

LULESH – Speedup Over Serial (Higher is Better)

Test Specs

2 Power8 sockets @ 4GHz (8 cores,
with 8 threads each) with 1 NVIDIA
Pascal P100 GPU.

Compiler Options: -O3 -qhot
-qsmp=omp -qoffload*
* Where applicable

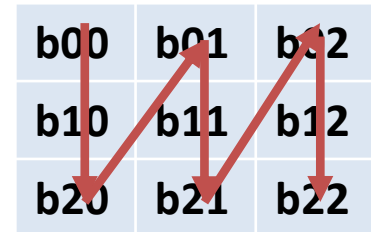


Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```

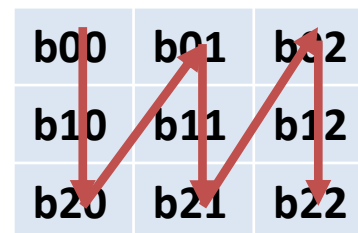


Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

Cache Misses: 0

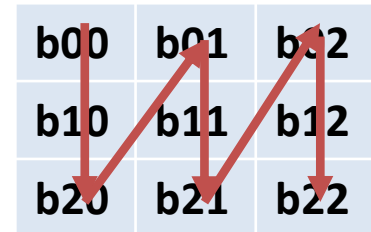
i	j	k
0	0	0

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

Cache Misses: 1

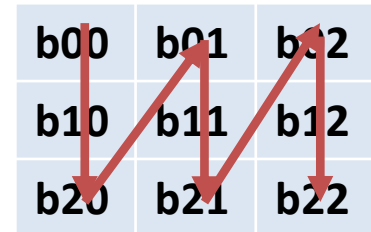
i	j	k
0	0	1

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

Cache Misses: 1

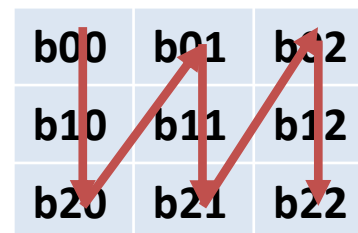
i	j	k
0	0	1

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

Cache Misses: 2

i	j	k
0	0	2

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```

b00	b01	b02
b10	b11	b12
b20	b21	b22

- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

Cache Misses: 2

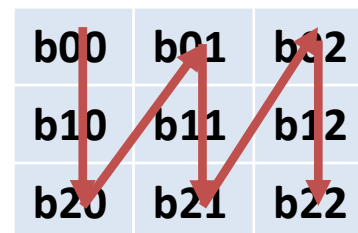
i	j	k
0	0	2

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

Cache Misses: 3

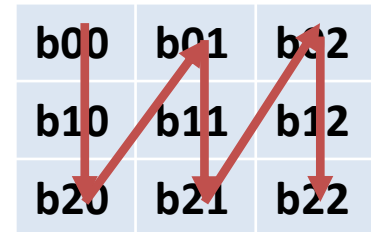
i	j	k
0	1	0

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

Cache Misses: 3

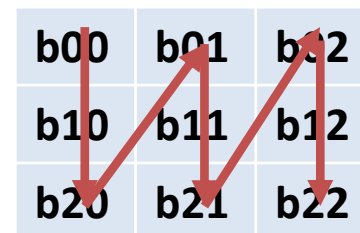
i	j	k
0	1	0

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

Cache Misses: 4

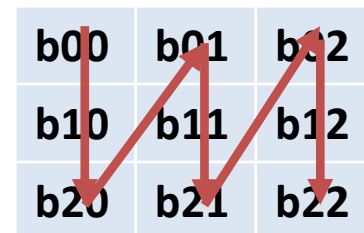
i	j	k
0	1	1

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

Cache Misses: 4

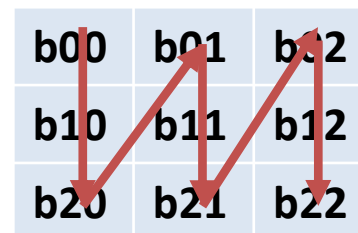
i	j	k
0	1	1

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

Cache Misses: 5

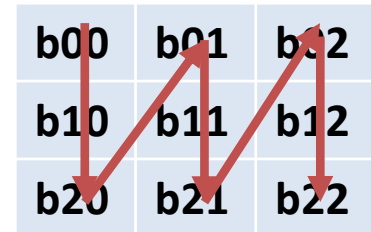
i	j	k
0	1	2

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

Cache Misses: 5

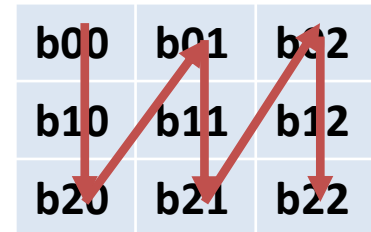
i	j	k
0	1	2

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

Cache Misses: 6

i	j	k
0	2	0

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```

b00	b01	b02
b10	b11	b12
b20	b21	b22

- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

Cache Misses: 6

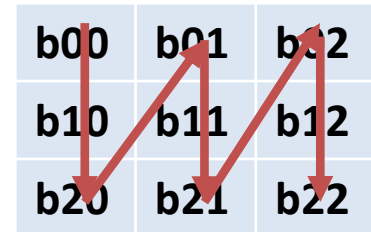
i	j	k
0	2	0

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

Cache Misses: 7

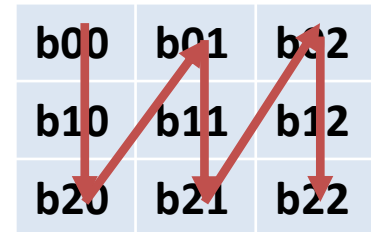
i	j	k
0	2	1

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

Cache Misses: 7

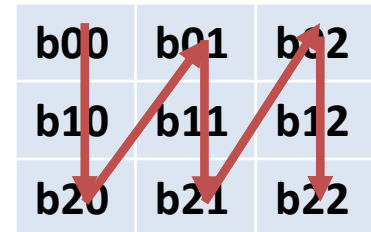
i	j	k
0	2	1

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

Cache Misses: 8

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

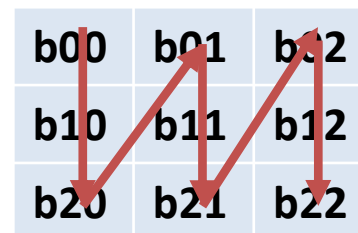
i	j	k
0	2	2

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



- Processors **load data into their cache** for fast reuse
- They always load a bunch of data at the same time
- They do **branch-prediction** and **pre-fetching**!

However here we are not helping him...

Processor cache

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

Cache Misses: 8

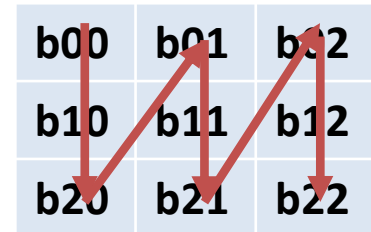
i	j	k
0	2	2

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

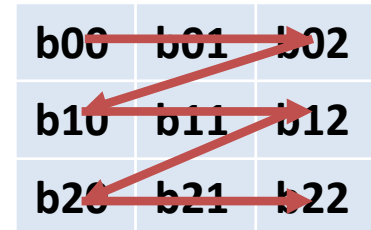
The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



The better way (row-major):

```
for(int i=0; i<n; ++i)
  for(int k=0; k<n; ++k)
    for(int j=0; j<n; ++j)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



Let's go...

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

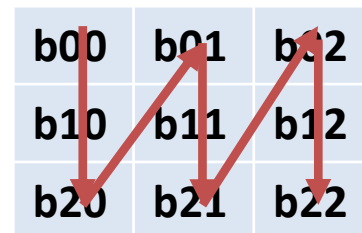
i	k	j
0	0	0

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



The better way (row-major):

```
for(int i=0; i<n; ++i)
  for(int k=0; k<n; ++k)
    for(int j=0; j<n; ++j)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



Cache Misses: 0

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

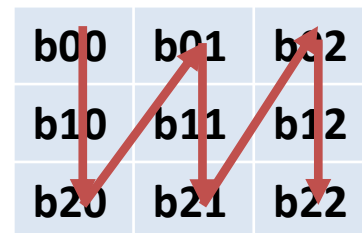
i	k	j
0	0	1

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



The better way (row-major):

```
for(int i=0; i<n; ++i)
  for(int k=0; k<n; ++k)
    for(int j=0; j<n; ++j)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



Cache Misses: 0

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

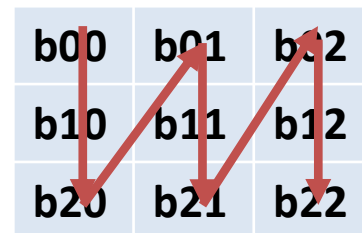
i	k	j
0	0	2

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



The better way (row-major):

```
for(int i=0; i<n; ++i)
  for(int k=0; k<n; ++k)
    for(int j=0; j<n; ++j)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



Cache Misses: 1

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

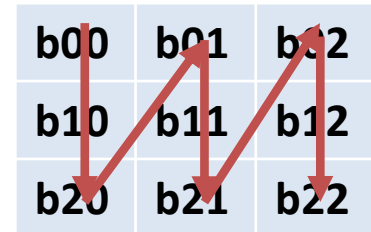
i	k	j
0	1	0

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

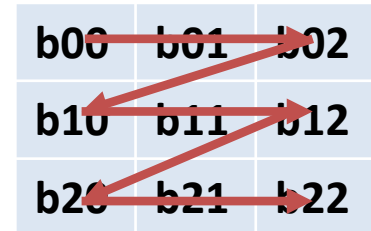
The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



The better way (row-major):

```
for(int i=0; i<n; ++i)
  for(int k=0; k<n; ++k)
    for(int j=0; j<n; ++j)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



Cache Misses: 1

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

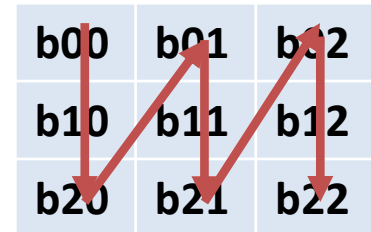
i	k	j
0	1	0

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

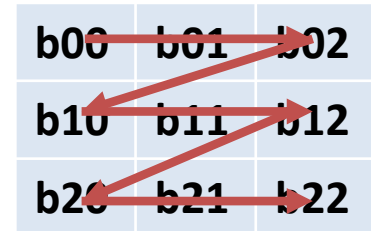
The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



The better way (row-major):

```
for(int i=0; i<n; ++i)
  for(int k=0; k<n; ++k)
    for(int j=0; j<n; ++j)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



Cache Misses: 1

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

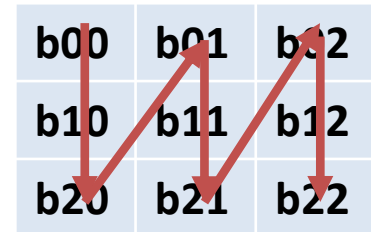
i	k	j
0	1	1

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

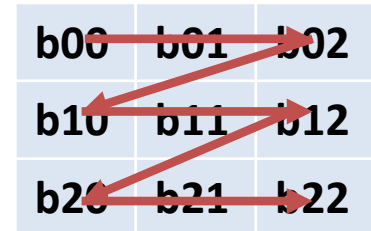
The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



The better way (row-major):

```
for(int i=0; i<n; ++i)
  for(int k=0; k<n; ++k)
    for(int j=0; j<n; ++j)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



Cache Misses: 1

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

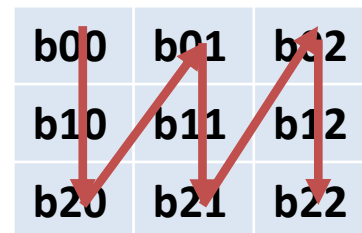
i	k	j
0	1	2

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



The better way (row-major):

```
for(int i=0; i<n; ++i)
  for(int k=0; k<n; ++k)
    for(int j=0; j<n; ++j)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



Cache Misses: 2

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

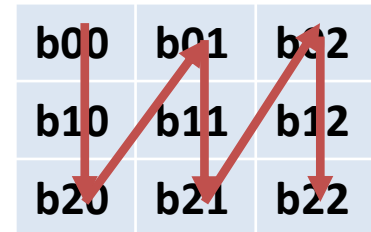
i	k	j
0	2	0

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

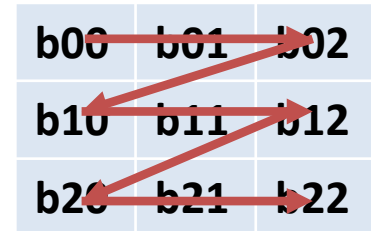
The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



The better way (row-major):

```
for(int i=0; i<n; ++i)
  for(int k=0; k<n; ++k)
    for(int j=0; j<n; ++j)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



Cache Misses: 2

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

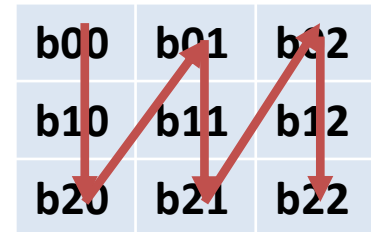
i	k	j
0	2	0

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

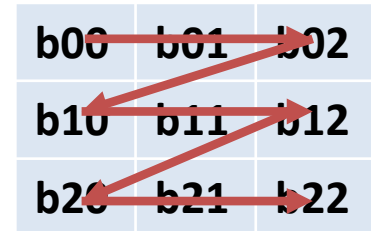
The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



The better way (row-major):

```
for(int i=0; i<n; ++i)
  for(int k=0; k<n; ++k)
    for(int j=0; j<n; ++j)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



Cache Misses: 2

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

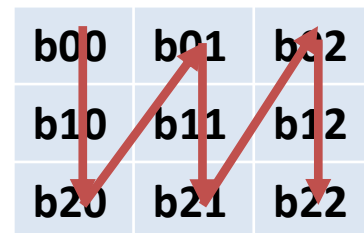
i	k	j
0	2	1

Full example: matrix multiplication

Sub-optimal memory access will kill the performance...

The less efficient way (column-major):

```
for(int i=0; i<n; ++i)
  for(int j=0; j<n; ++j)
    for(int k=0; k<n; ++k)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



The better way (row-major):

```
for(int i=0; i<n; ++i)
  for(int k=0; k<n; ++k)
    for(int j=0; j<n; ++j)
      c[i*n+j] += a[i*n+k] * b[k*n+j];
```



Cache Misses: 2

a00	a01	a02	a10	a11	a12	a20	a21	a22
b00	b01	b02	b10	b11	b12	b20	b21	b22
c00	c01	c02	c10	c11	c12	c20	c21	c22

i	k	j
0	2	2

Full example: matrix multiplication with OpenMP / Numba

```
#pragma omp parallel for
for(int i=0; i<size; ++i)
    for(int k=0; k<size; ++k)
        for(int j=0; j<size; ++j)
            c[i*size+j] += a[i*size+k] * b[k*size+j];
```

C++

```
!$omp parallel private(i,j,k)
!$omp do schedule(static)
do i=0,size-1
    do k=0,size-1
        do j=0,size-1
            c(i*size+j) = c(i*size+j) + a(i*size+k) * b(k*size+j)
        enddo
    enddo
enddo
!$omp end do
!$omp end parallel
```

FORTRAN

```
@jit(Parallel=True)
def mat_mul(a,b,c,size):
    for i in prange(size):
        for k in range(size):
            for j in range(size):
                c[i*size+j] += a[i*size+k] * b[k*size+j]
```

Python

Full example: matrix multiplication with GPU offloading (OpenMP 4.5+)

```
!$omp target map(tofrom:c) map(to:a,b)
!$omp teams distribute parallel do private(i,j,k) collapse(2)
do i=0,size-1
  do j=0,size-1
    do k=0,size-1
      c(i*size+j) = c(i*size+j) + a(i*size+k) * b(k*size+j)
    enddo
  enddo
enddo
!$omp end target
```

FORTRAN

```
#pragma omp target map(to: a[0:n2], b[0:n2]), map(tofrom: c[0:n2])
#pragma omp teams distribute parallel for collapse(2)
for(int i=0; i<size; ++i)
  for(int j=0; j<size; ++j)
    for (int k = 0; k < size; k++)
      c[i*size+j] += a[i*size+k] * b[k*size+j];
```

C++

Full example: matrix multiplication with CUDA

```
__global__ void cuda_mul(float* a, float* b, float* c, int size) {  
    int row = blockIdx.y*blockDim.y+threadIdx.y;  
    int col = blockIdx.x*blockDim.x+threadIdx.x;  
    for (int i = 0; i < size; i++)  
        c[row*size+col] += a[row*size+i] * b[i*size+col];  
}
```

CUDA Kernel

```
cudaMemcpy(dm1, a, sizeof(float)*size*size, cudaMemcpyHostToDevice);  
cudaMemcpy(dm2, b, sizeof(float)*size*size, cudaMemcpyHostToDevice);  
cudaMemcpy(dm3, c, sizeof(float)*size*size, cudaMemcpyHostToDevice);  
  
dim3 blockSize = dim3(16, 16);  
dim3 gridSize = dim3(size / blockSize.x, size / blockSize.y);  
  
cuda_mul<<<gridSize, blockSize>>>(dm1, dm2, dm3, size);  
  
cudaMemcpy(c, dm3, sizeof(float)*size*size, cudaMemcpyDeviceToHost);
```

Kernel Call

Full example: matrix multiplication with CUDA

```
__global__ void cuda_mul(float* a, float* b, float* c, int size) {
    int row = blockIdx.y*blockDim.y+threadIdx.y;
    int col = blockIdx.x*blockDim.x+threadIdx.x;
    for (int i = 0; i < size; i++)
        c[row*size+col] += a[row*size+i] * b[i*size+col];
}
```

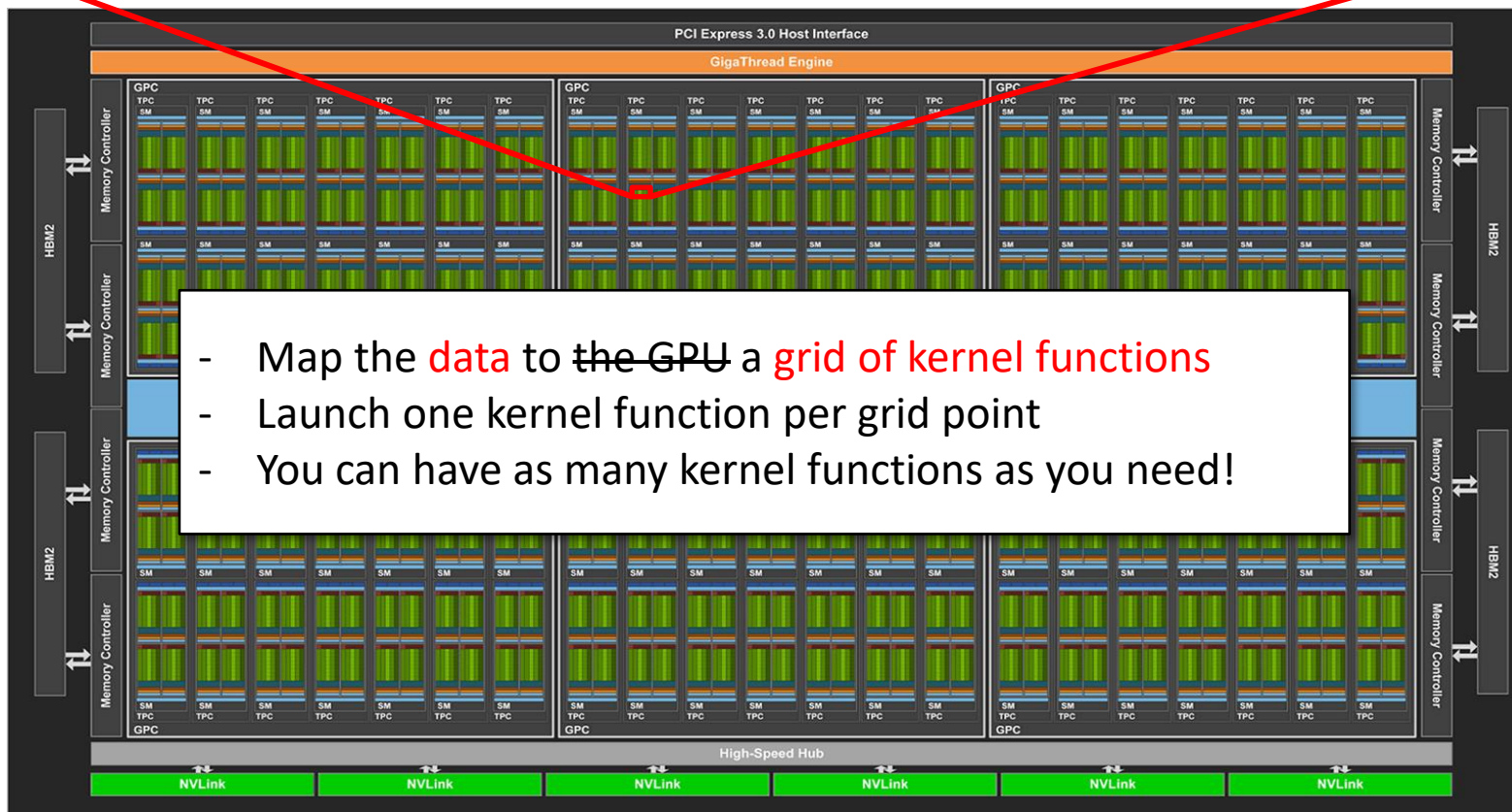
CUDA Kernel



Full example: matrix multiplication with CUDA

```
__global__ void cuda_mul(float* a, float* b, float* c, int size) {  
    int row = blockIdx.y*blockDim.y+threadIdx.y;  
    int col = blockIdx.x*blockDim.x+threadIdx.x;  
    for (int i = 0; i < size; i++)  
        c[row*size+col] += a[row*size+i] * b[i*size+col];  
}
```

CUDA Kernel



Full example: matrix multiplication with CUDA and Numba

```
@cuda.jit
def mat_mul(a,b,c,size):
    row=cuda.blockIdx.y*cuda.blockDim.y+cuda.threadIdx.y
    col=cuda.blockIdx.x*cuda.blockDim.x+cuda.threadIdx.x
    for i in range(size):
        c[row*size+col] += a[row*size+i] * b[i*size+col]
```

CUDA Kernel

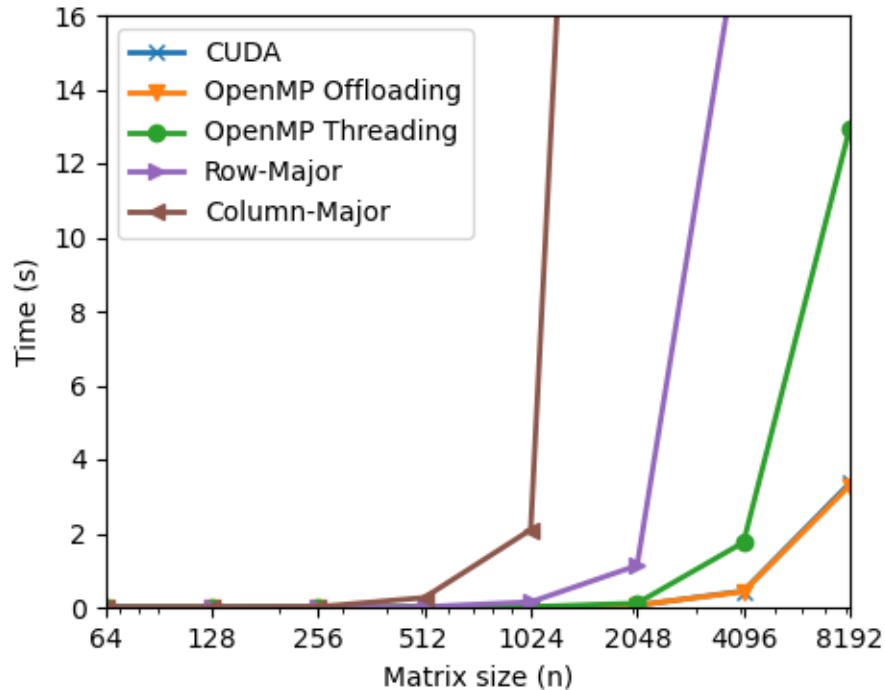
```
threadsperblock = (16,16)
blockspergrid_x = int(np.ceil(size_array / threadsperblock[0]))
blockspergrid_y = int(np.ceil(size_array / threadsperblock[1]))
blockspergrid = (blockspergrid_x, blockspergrid_y)

mat_mul[blockspergrid, threadsperblock](a,b,c,size_array)
```

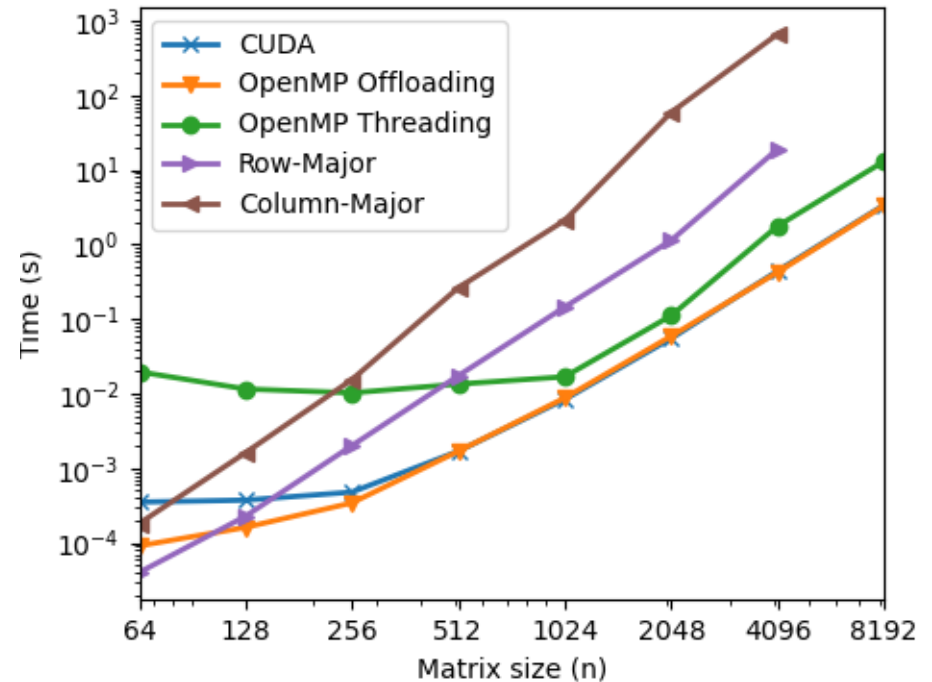
Kernel Call

Full example: matrix multiplication performance

Linear / log scale



Log / log scale



N = 4096

Column-Major: 658s

OpenMP 4.5/CUDA GPU: 0.43s

=> 1500x performance speedup!

There are two main ways of parallelizing a code



Open Multi-Processing



Message Passing Interface

Both are APIs for C/C++ and Fortran

Works in **shared** memory systems

Can use as many cores as there are in the **compute node**

Uses **threads** to split the work

Threads have both **private and shared** variables

Uses **preprocessor directives** in commented lines

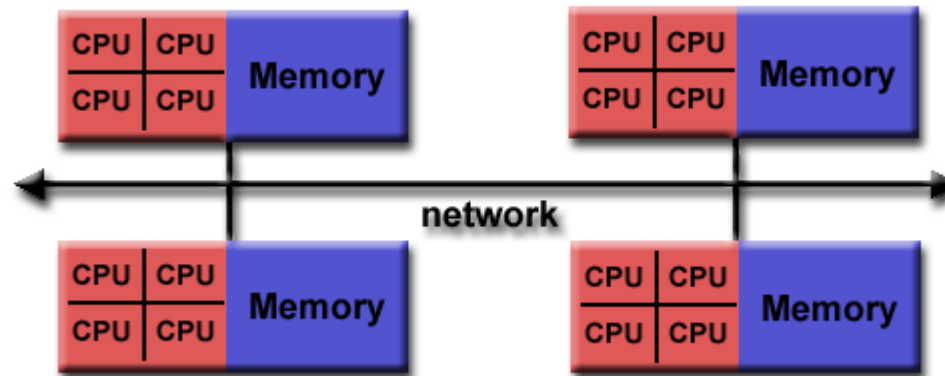
Works in **shared and distributed** memory systems

Can use as many cores as there are in the **cluster**

Uses **processes** to split the work

Processes have only **private** variables

Loads a library and uses **specific commands**



There are two main ways of parallelizing a code



Open Multi-Processing

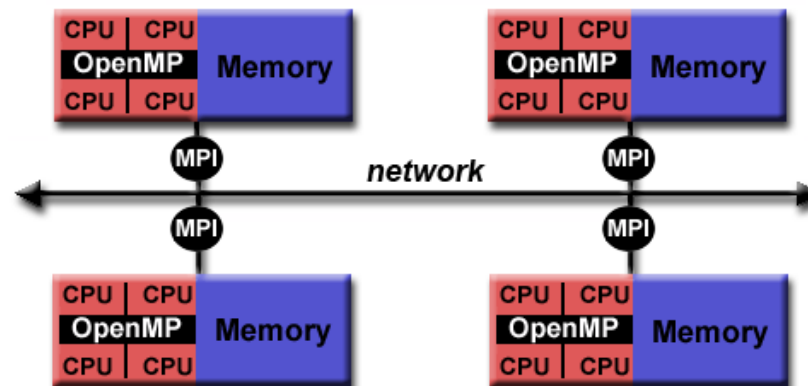


Message Passing Interface

Both are APIs for C/C++ and Fortran

Works in **shared** memory systems
Can use as many cores as there are in the **compute node**
Uses **threads** to split the work
Threads have both **private and shared** variables
Uses **preprocessor directives** in commented lines

Works in **shared and distributed** memory systems
Can use as many cores as there are in the **cluster**
Uses **processes** to split the work
Processes have only **private** variables
Loads a library and uses **specific commands**



What about MPI?

Loads a library and uses **specific commands**

```
include 'mpif.h'

<variable declarations>

call MPI_INIT(ierr)
call MPI_COMM_RANK(MPI_COMM_WORLD,id,ierr)
call MPI_COMM_SIZE(MPI_COMM_WORLD,nproc,ierr)

<calculations>

call MPI_FINALIZE(ierr)
```



What about MPI?

Loads a library and uses **specific commands**

```
include 'mpif.h'

<variable declarations>

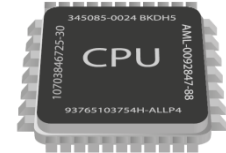
call MPI_INIT(ierr)
call MPI_COMM_RANK(MPI_COMM_WORLD,id,ierr)
call MPI_COMM_SIZE(MPI_COMM_WORLD,nproc,ierr)

do i=1,N
  <calculations>
enddo

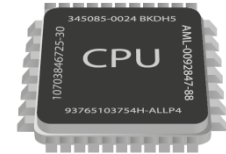
call MPI_FINALIZE(ierr)
```

N iterations

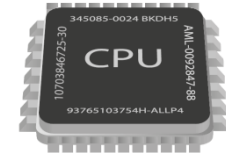
$N/4$



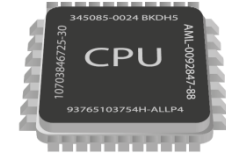
$N/4$



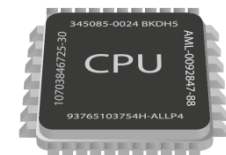
$N/4$



$N/4$



...



What about MPI?

Loads a library and uses **specific commands**

```
include 'mpif.h'

<variable declarations>

call MPI_INIT(ierr)
call MPI_COMM_RANK(MPI_COMM_WORLD,id,ierr)
call MPI_COMM_SIZE(MPI_COMM_WORLD,nproc,ierr)

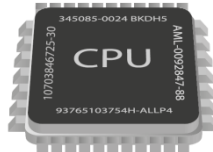
do i=ini,fin
  <calculations>
enddo

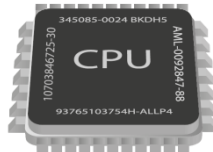
call MPI_FINALIZE(ierr)
```

N iterations

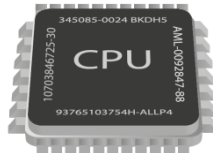
$N/nproc$ →  0

$N/nproc$ →  1

$N/nproc$ →  2

$N/nproc$ →  3

...

$N/nproc$ →  **nproc-1**



What about MPI?

Loads a library and uses **specific commands**

```
include 'mpif.h'

<variable declarations>

call MPI_INIT(ierr)
call MPI_COMM_RANK(MPI_COMM_WORLD,id,ierr)
call MPI_COMM_SIZE(MPI_COMM_WORLD,nproc,ierr)

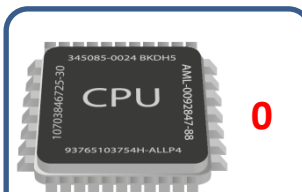
ini=id*(N/nproc)+1
fin=(id+1)*(N/nproc)

do i=ini,fin
  <calculations>
enddo

call MPI_FINALIZE(ierr)
```

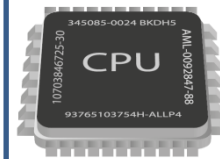
N iterations

$N/nproc$ →



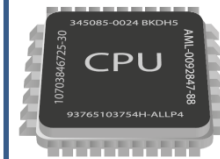
0

$N/nproc$ →



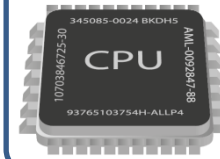
1

$N/nproc$ →



2

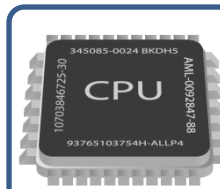
$N/nproc$ →



3

...

$N/nproc$ →



$nproc-1$



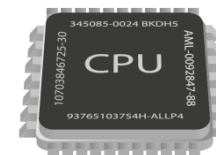
OPEN MPI

--	--	--	--	--	--	--	--	--	--	--	--

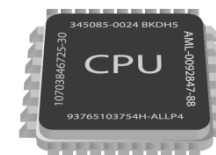
--	--	--	--	--	--	--	--	--	--	--	--

--	--	--	--	--	--	--	--	--	--	--	--

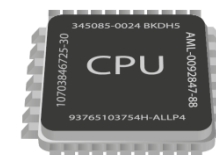
--	--	--	--	--	--	--	--	--	--	--	--



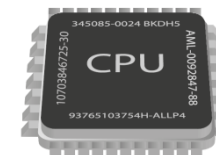
0



1

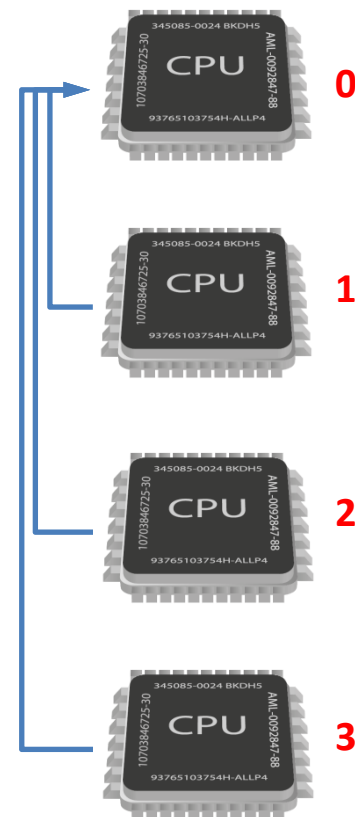
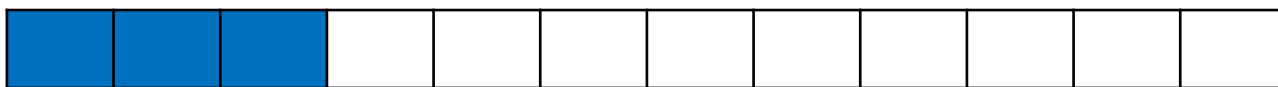


2

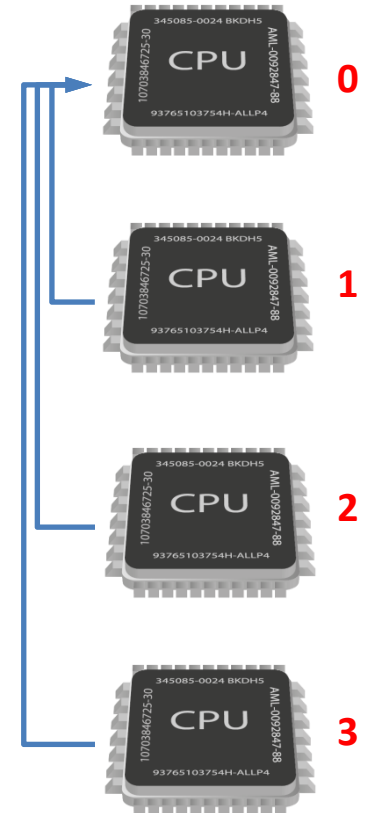
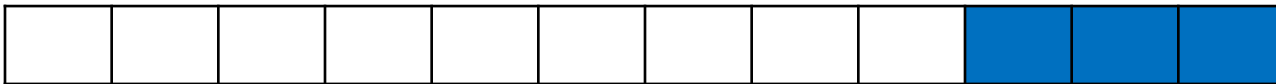


3





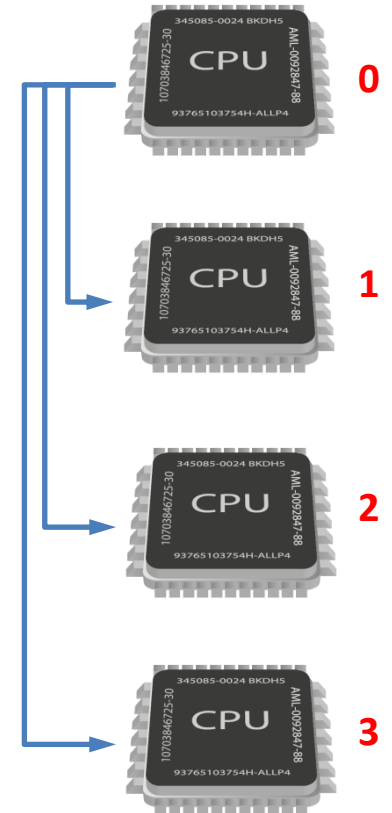
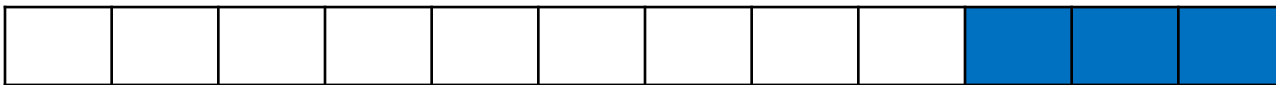
Only process 0 has the correct values!



```
call MPI_REDUCE(arr,result,arraysize,<type>,<operation>,<dest_id>,MPI_WORLD_COMM,ierr)
call MPI_REDUCE(arr,result,12,MPI_INT,MPI_SUM,0,MPI_WORLD_COMM,ierr)
```



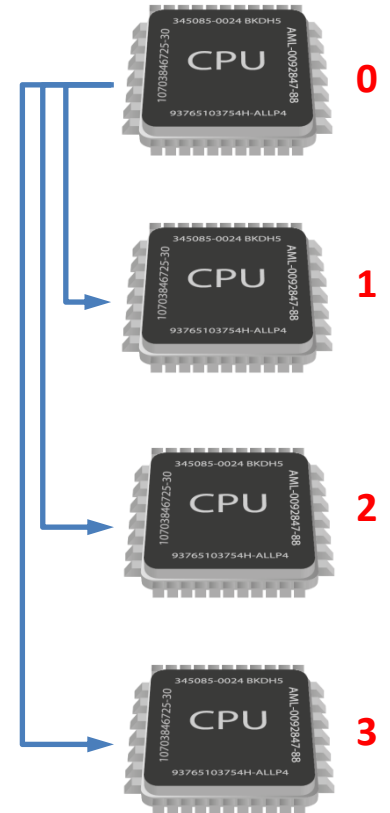
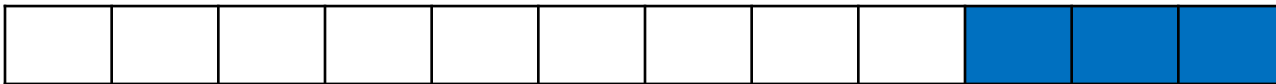
Only process 0 has the correct values!



```
call MPI_REDUCE(arr,result,arraysize,<type>,<operation>,<dest_id>,MPI_WORLD_COMM,ierr)
call MPI_REDUCE(arr,result,12,MPI_INT,MPI_SUM,0,MPI_WORLD_COMM,ierr)
```



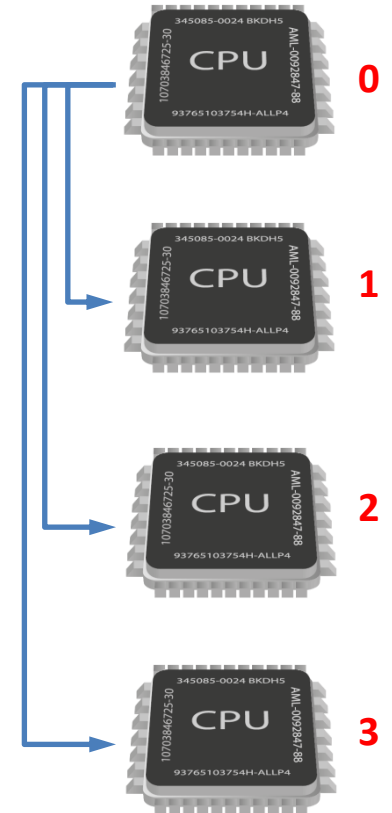
Only process 0 has the correct values!



```
call MPI_REDUCE(arr,result,arraysize,<type>,<operation>,<dest_id>,MPI_WORLD_COMM,ierr)
call MPI_REDUCE(arr,result,12,MPI_INT,MPI_SUM,0,MPI_WORLD_COMM,ierr)
call MPI_BCAST(result,12,MPI_INT,0,MPI_WORLD,COMM,ierr)
```

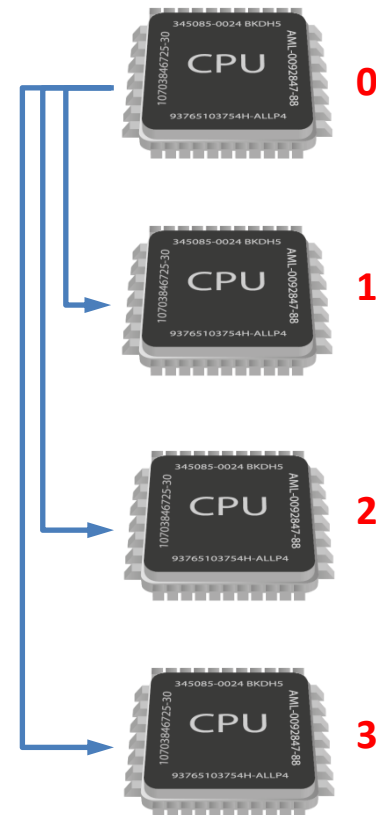


Only process 0 has the correct values!



```
call MPI_REDUCE(arr,result,arraysize,<type>,<operation>,<dest_id>,MPI_WORLD_COMM,ierr)
call MPI_REDUCE(arr,result,12,MPI_INT,MPI_SUM,0,MPI_WORLD_COMM,ierr)
call MPI_BCAST(result,12,MPI_INT,0,MPI_WORLD_COMM,ierr)
```





```
call MPI_ALLREDUCE(arr,result,12,MPI_INT,MPI_SUM,MPI_WORLD_COMM,ierr)
```



Example: Numerical integration

```
include 'mpif.h'
```

```
call MPI_INIT(ierr); comm = MPI_COMM_WORLD
```

```
call MPI_COMM_RANK(comm, rank, ierr)
```

```
call MPI_COMM_SIZE(comm, size, ierr)
```

Initialize MPI

FORTRAN



Example: Numerical integration

```
include 'mpif.h'

call MPI_INIT(ierr); comm = MPI_COMM_WORLD
call MPI_COMM_RANK(comm, rank, ierr)
call MPI_COMM_SIZE(comm, size, ierr)

totalsteps = 1000000
steps = totalsteps/size

ini = rank * steps + 1
fin = (rank+1) * steps

if(fin.ge.totalsteps) then
    fin = totalsteps
endif
```

Initialize MPI

Define the range

FORTRAN



Example: Numerical integration

```
include 'mpif.h'

call MPI_INIT(ierr); comm = MPI_COMM_WORLD
call MPI_COMM_RANK(comm, rank, ierr)
call MPI_COMM_SIZE(comm, size, ierr)

totalsteps = 1000000
steps = totalsteps/size

ini = rank * steps + 1
fin = (rank+1) * steps

if(fin.ge.totalsteps) then
    fin = totalsteps
endif

dx = 1.d0 / totalsteps
x = -0.5d0 * dx

localSum = 0.d0
do i = ini, fin
    x = (i-0.5d0)*dx
    localSum = localSum + 4.d0 / (1.d0 + x*x)
enddo
localSum = localSum * dx
```

Initialize MPI

Define the range

Do the computation

FORTRAN



Example: Numerical integration

```
include 'mpif.h'
```

```
call MPI_INIT(ierr); comm = MPI_COMM_WORLD  
call MPI_COMM_RANK(comm, rank, ierr)  
call MPI_COMM_SIZE(comm, size, ierr)
```

Initialize MPI

```
totalsteps = 1000000  
steps = totalsteps/size
```

```
ini = rank * steps + 1  
fin = (rank+1) * steps
```

Define the range

```
if(fin.ge.totalsteps) then  
    fin = totalsteps  
endif
```

```
dx = 1.d0 / totalsteps  
x = -0.5d0 * dx
```

```
localSum = 0.d0  
do i = ini, fin  
    x = (i-0.5d0)*dx  
    localSum = localSum + 4.d0 / (1.d0 + x*x)  
enddo  
localSum = localSum * dx
```

Do the computation

```
call MPI_ALLREDUCE(localSum,globalSum,1,MPI_DOUBLE_PRECISION,MPI_SUM,comm,ierr)
```

```
write(*,*) localSum, globalSum
```

Sum the data across nodes

FORTRAN



Example: Numerical integration

```
include 'mpif.h'
```

```
call MPI_INIT(ierr); comm = MPI_COMM_WORLD  
call MPI_COMM_RANK(comm, rank, ierr)  
call MPI_COMM_SIZE(comm, size, ierr)
```

Initialize MPI

```
totalsteps = 1000000  
steps = totalsteps/size
```

Define the range

```
ini = rank * steps + 1  
fin = (rank+1) * steps
```

```
if(fin.ge.totalsteps) then  
    fin = totalsteps  
endif
```

```
dx = 1.d0 / totalsteps  
x = -0.5d0 * dx
```

```
localSum = 0.d0  
do i = ini, fin  
    x = (i-0.5d0)*dx  
    localSum = localSum + 4.d0 / (1.d0 + x*x)  
enddo  
localSum = localSum * dx
```

```
call MPI_ALLREDUCE(localSum,globalSum,1,MPI_DOUBLE_PRECISION,MPI_SUM,comm,ierr)
```

```
write(*,*) localSum, globalSum
```

```
call MPI_FINALIZE(ierr)
```

Don't forget to Finalize MPI

FORTRAN



Example: Numerical integration

```
from mpi4py import MPI
import numpy

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

totalsteps = 1000000
steps = totalsteps/size

ini = rank * steps
fin = (rank+1) * steps

if fin > totalsteps:
    fin = totalsteps

dx = 1./totalsteps
x = -0.5*dx

localSum = 0.
for i in range(int(ini), int(fin)):
    x = (i-0.5)*dx
    localSum = localSum + 4./(1. + x*x)

localSum = localSum * dx

globalSum = comm.allreduce(localSum, op=MPI.SUM)

print(localSum, globalSum)
```

Initialize MPI

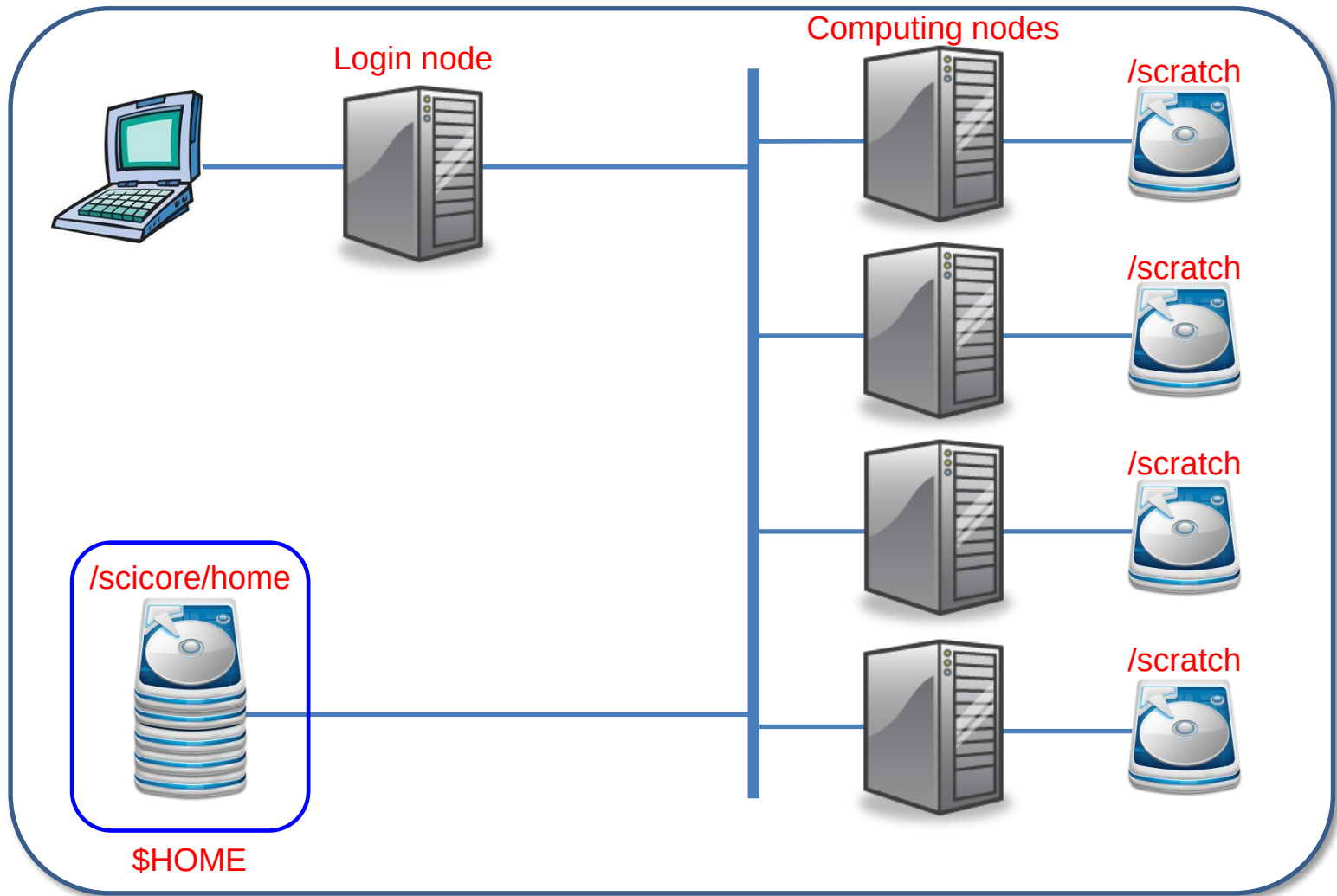
Define the range

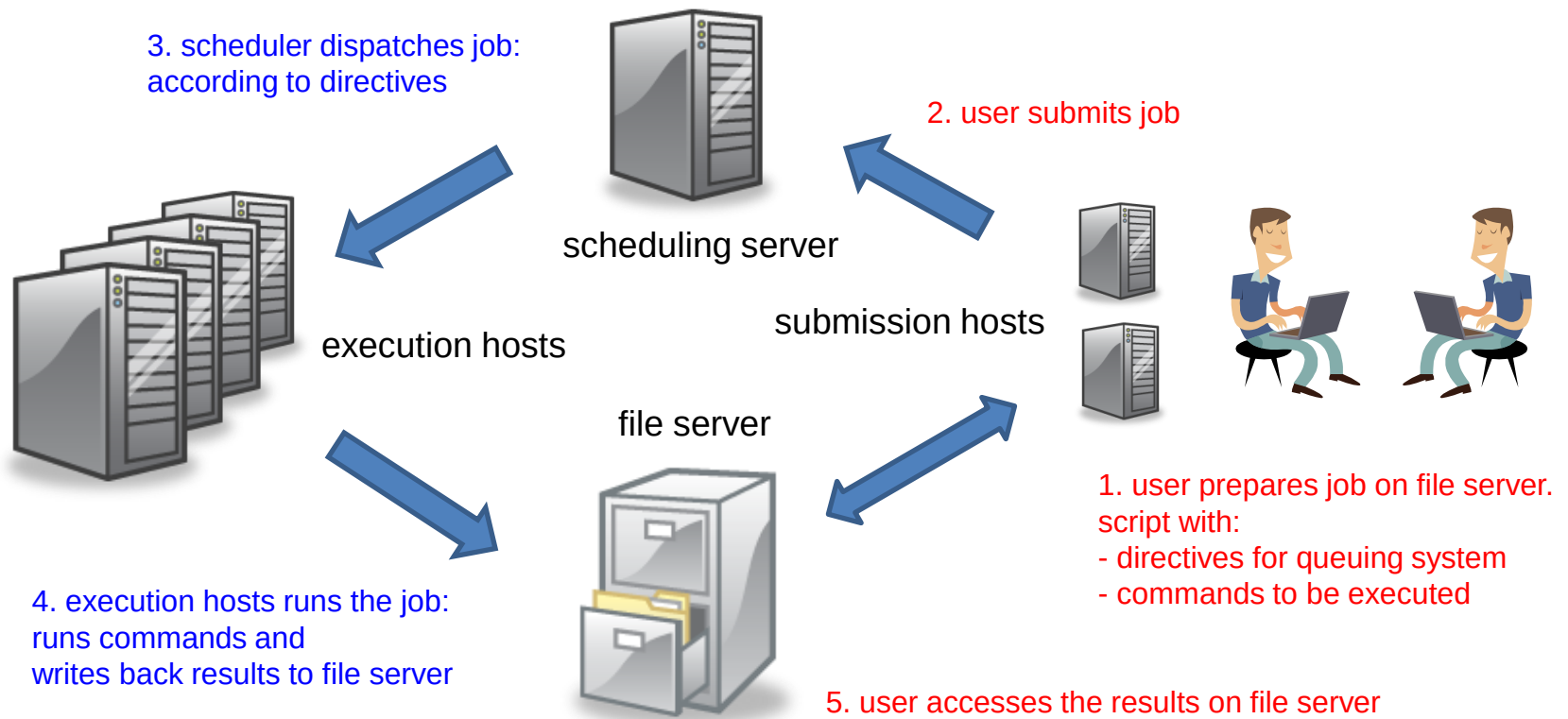
Python

Do the computation

Sum the data across nodes







- User takes care
- Cluster takes care

Lest's play "Spot the Differences"

Lest's play "Spot the Differences"

Which script launches an OpenMP job and which one an MPI?

```
#!/bin/bash
```

```
#SBATCH --job-name=myJob
#SBATCH --cpus-per-task=8
#SBATCH --mem=3.5G
#SBATCH --time=05:00:00
#SBATCH --qos=6hour
#SBATCH --output=/path/to/stdout/folder
#SBATCH --error=/path/to/stderr/folder
#SBATCH --mail-type=END,FAIL,TIME_LIMIT
#SBATCH --mail-user=mailaddress@unibas.ch
```

```
# Tell OpenMP how many threads to use
#####
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
```

```
# load your required modules
#####
module load <module name>
...
```

```
# and here goes your command line
./my_openmp_program
```

```
# Some third party programs include a threading
# option, configuration or environment variable
```

```
#!/bin/bash
```

```
#SBATCH --job-name=myJob
#SBATCH --ntasks=8
#SBATCH --mem=3.5G
#SBATCH --time=05:00:00
#SBATCH --qos=6hour
#SBATCH --output=/path/to/stdout/folder
#SBATCH --error=/path/to/stderr/folder
#SBATCH --mail-type=END,FAIL,TIME_LIMIT
#SBATCH --mail-user=mailaddress@unibas.ch
```

```
# Tell OpenMP how many threads to use
#####
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
```

```
# load your required modules
#####
module load <module name>
...
```

```
# and here goes your command line
srun ./my_openmp_program
```

```
# Some third party programs include a threading
# option, configuration or environment variable
```

```
#!/bin/bash
```

How do I launch an hybrid job?

```
#SBATCH --job-name=myJob
#SBATCH --ntasks=?
#SBATCH --tasks-per-node=?
#SBATCH --cpus-per-task=?
#SBATCH --mem=3.5G
#SBATCH --time=05:00:00
#SBATCH --qos=6hour
#SBATCH --output=/path/to/stdout/folder
#SBATCH --error=/path/to/stderr/folder
#SBATCH --mail-type=END,FAIL,TIME_LIMIT
#SBATCH --mail-user=mailaddress@unibas.ch
```

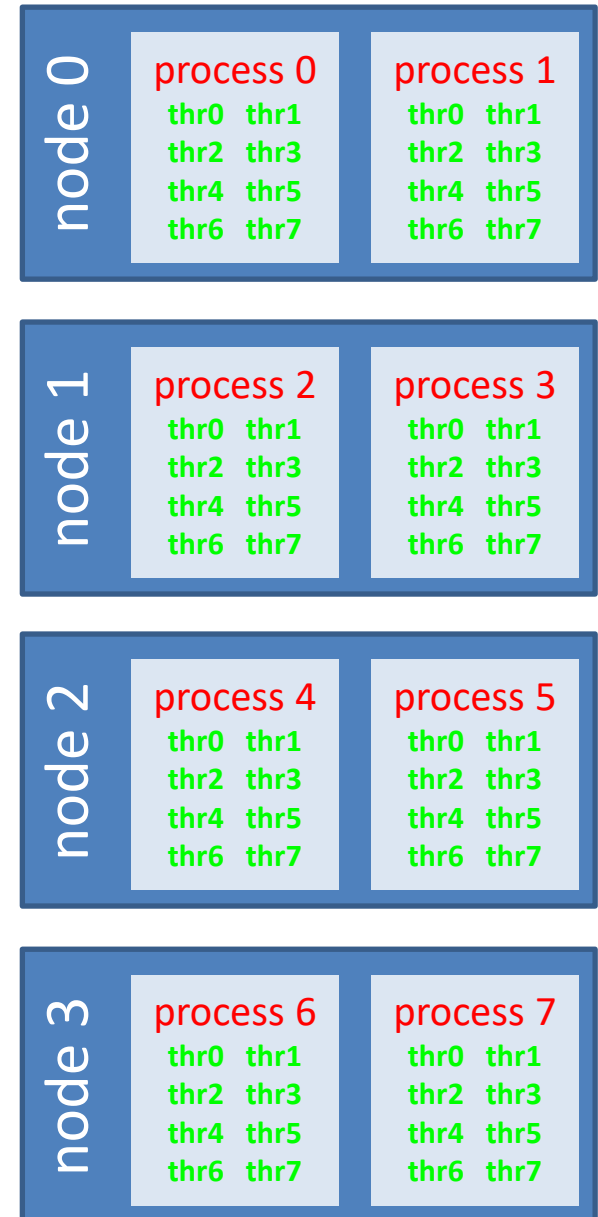
```
# Tell OpenMP how many threads to use
#####
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
```

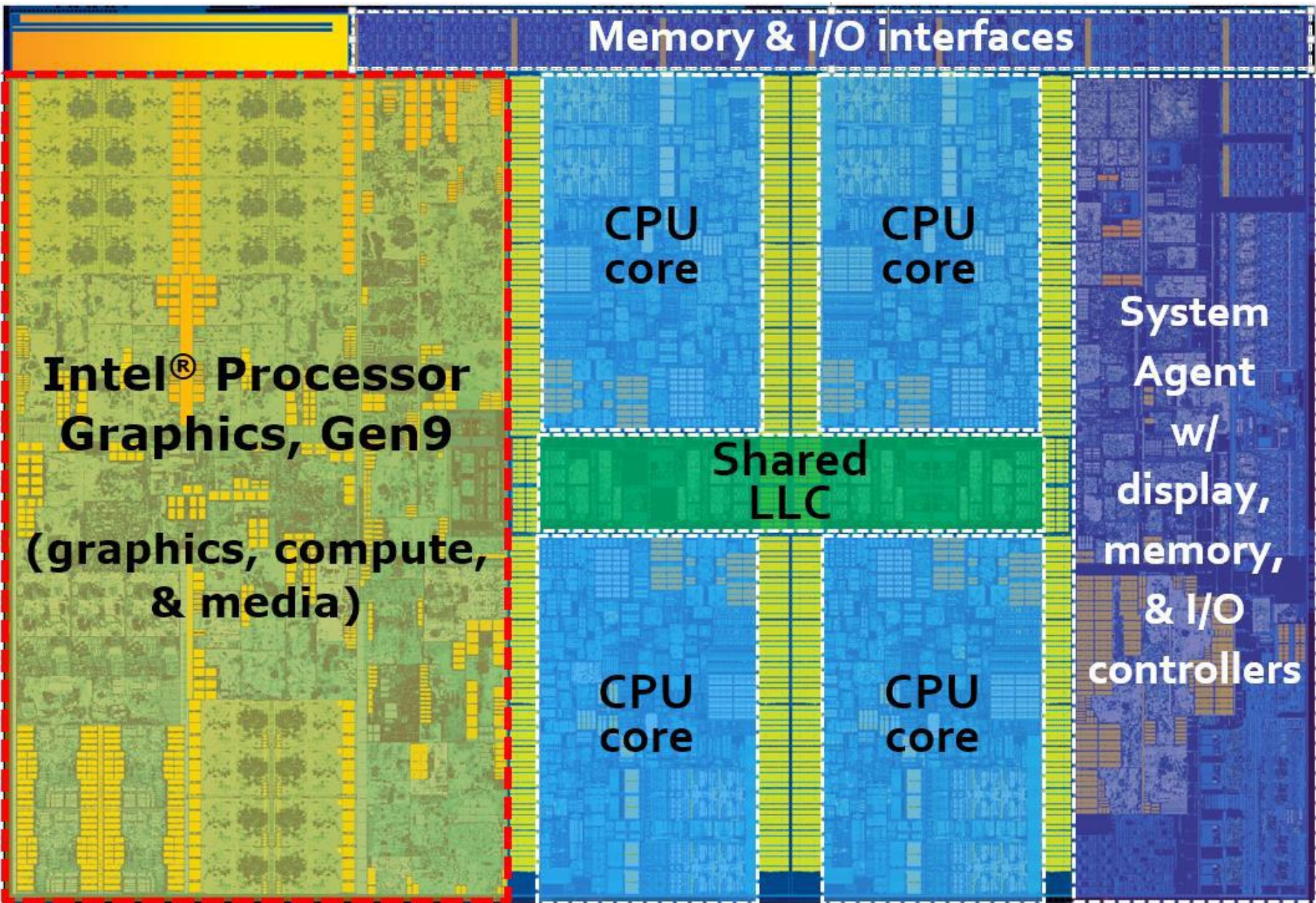
```
# load your required modules
#####
module load <module name>
```

...

```
# and here goes your command line
srun my_openmp_program
```

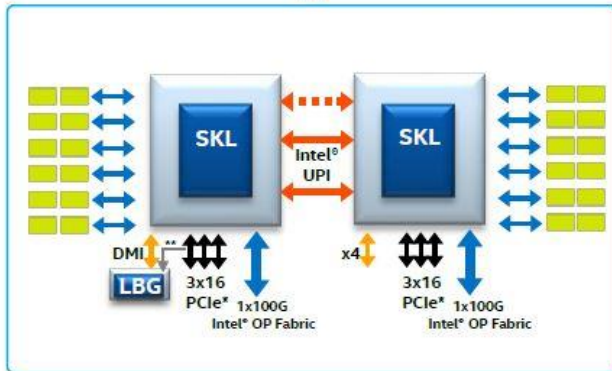
```
# Some third party programs include a threading
# option, configuration or environment variable
```





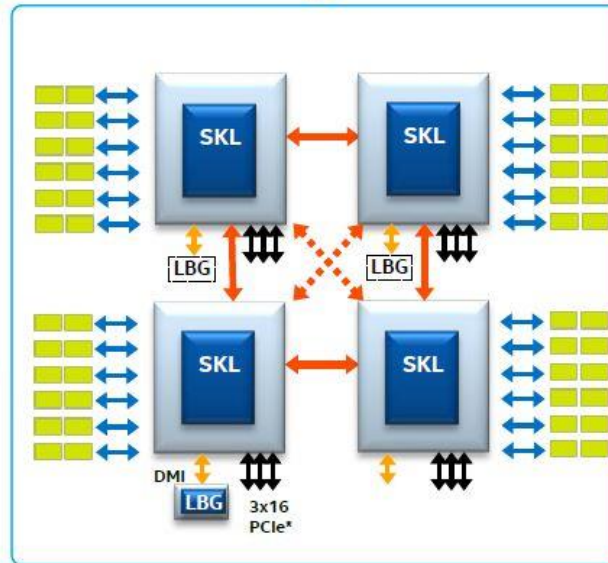
Platform Topologies

2S Configurations



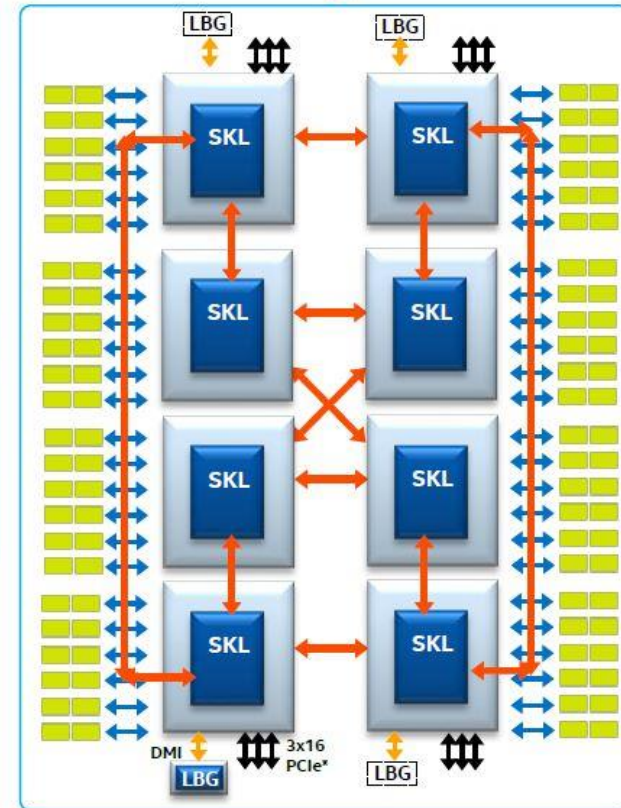
(2S-2UPI & 2S-3UPI shown)

4S Configurations



(4S-2UPI & 4S-3UPI shown)

8S Configuration



INTEL® XEON® SCALABLE PROCESSOR SUPPORTS CONFIGURATIONS RANGING FROM 2S-2UPI TO 8S

When using parallel programs you must be more aware of the hardware topology!

numactl --hardware

```
[cabezon@login10 ~]$ numactl --hardware
available: 2 nodes (0-1)
node 0 cpus: 0 1 2 3 4 5 6 7
node 0 size: 32742 MB
node 0 free: 188 MB
node 1 cpus: 8 9 10 11 12 13 14 15
node 1 size: 32768 MB
node 1 free: 10518 MB
node distances:
node  0  1
  0:  10  11
  1:  11  10
```

← Relative distances

Two sockets!

This is a NUMA (non-uniform memory access) system.

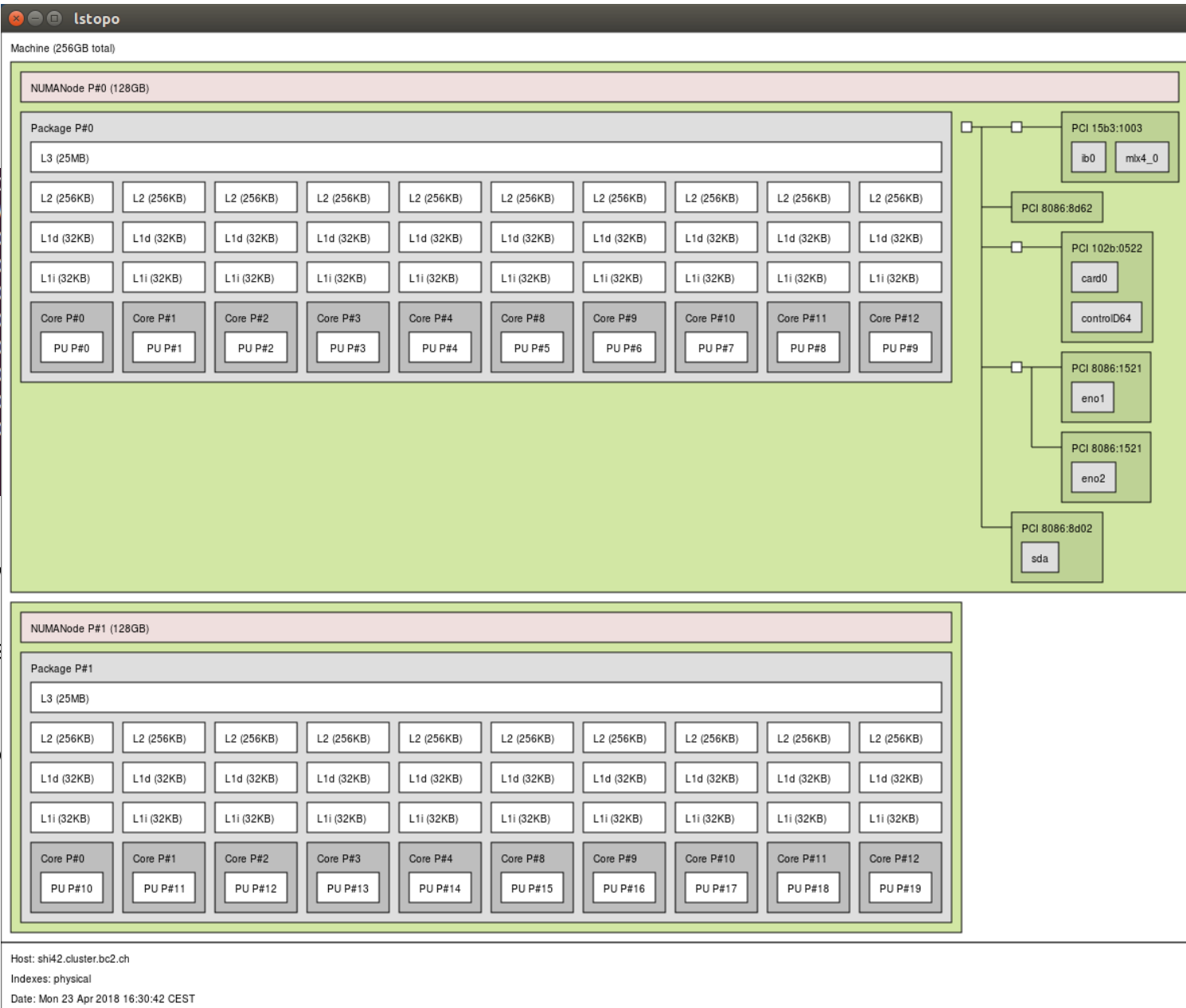
vs.

SMP (symmetric multi-processor):

```
ruben@jarvis:~$ numactl --hardware
available: 1 nodes (0)
node 0 cpus: 0 1 2 3 4 5 6 7
node 0 size: 7885 MB
node 0 free: 252 MB
node distances:
node  0
  0:  10
```

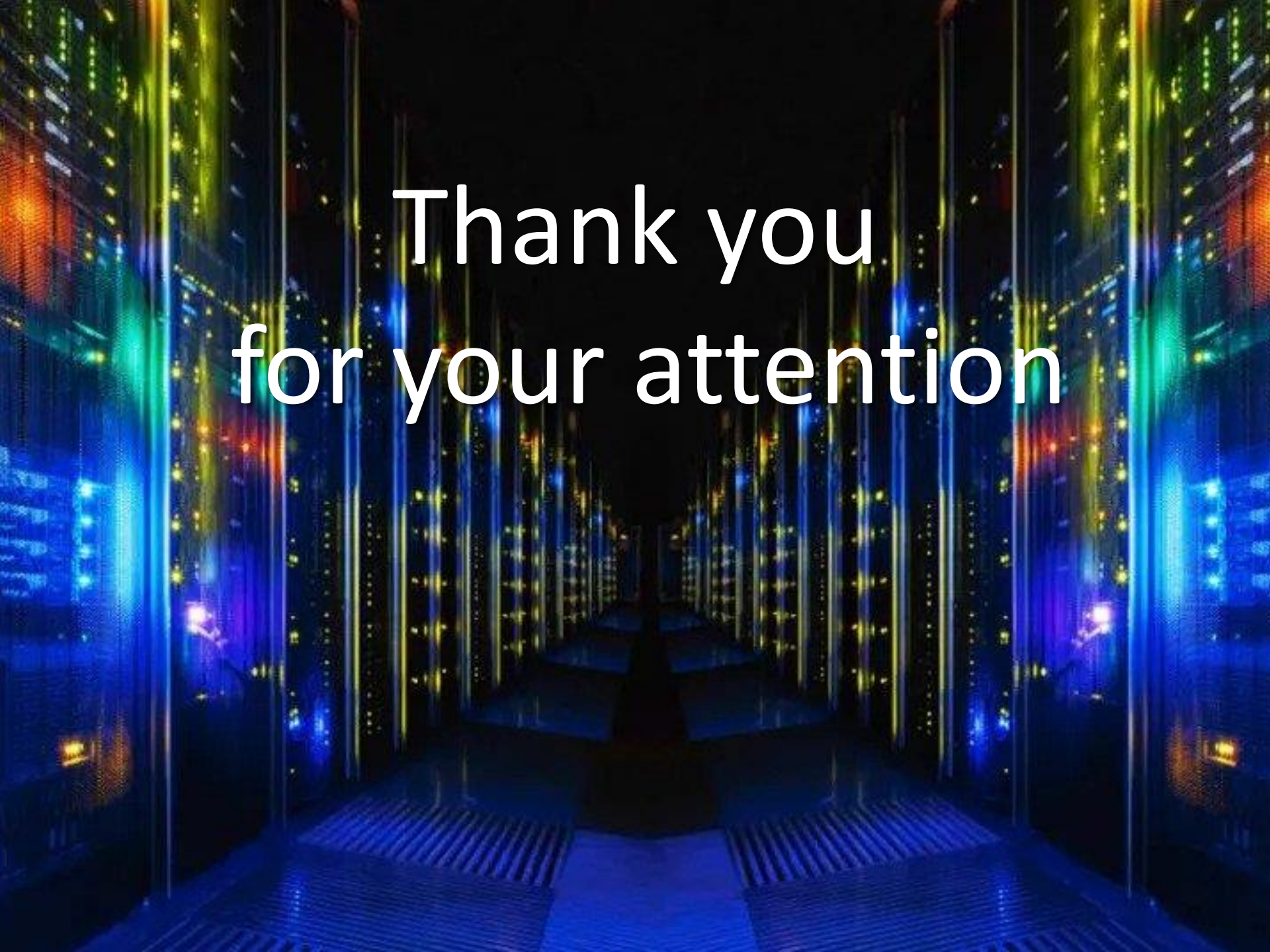
lscpu

```
[cabezon@login10 ~]$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                16
On-line CPU(s) list:   0-15
Thread(s) per core:    1
Core(s) per socket:    8
Socket(s):             2
NUMA node(s):          2
Vendor ID:             GenuineIntel
CPU family:            6
Model:                 45
Model name:            Intel(R) Xeon(R) CPU E5-2670 0 @ 2.60GHz
Stepping:              7
CPU MHz:               1298.578
BogoMIPS:              5205.23
Virtualization:        VT-x
L1d cache:             32K
L1i cache:             32K
L2 cache:              256K
L3 cache:              20480K
NUMA node0 CPU(s):     0-7
NUMA node1 CPU(s):     8-15
```



lstopo
(from hwloc package)

E5-2670 0 @ 2.60GHz



Thank you
for your attention